Pontificia Universidad Javeriana
Facultad de Ingeniería — Ingeniería de Sistemas

Software Architecture Document
SAD

GammaLab v2.0

Proyecto	Plataforma de análisis y visualización de señales neurofisiológicas Gamma Lab
Documento	Software Architecture Document — SAD
Versión	1.0
Estado	En elaboración
Fecha	27 de agosto de 2026

Autores

Juan Felipe Romero Ortiz — `juanromeroo@javeriana.edu.co`
Laura Juliana Garzón Arias — `garzonlj@javeriana.edu.co`
Xiomara Herrera Suárez — `x_herrera@javeriana.edu.co`
Sebastián Almanza Galvis — `s.almanzag@javeriana.edu.co`

Nota de uso. Documento académico/interno del equipo de proyecto. No distribuir sin autorización.
Trazabilidad. Este SAD define la arquitectura del sistema y traza su relación con el SRS de GammaLab v2.0.

Índice

1. Introducción **6**

1.1. Propósito . . . 6

1.2. Alcance . . . 6

1.3. Definiciones y acrónimos . . . 6

1.4. Audiencia . . . 7

1.5. Referencias relacionadas . . . 7

2. Contexto del sistema **7**

2.1. Visión general . . . 7

2.2. Límites del sistema . . . 8

2.2.1. Dentro del alcance . . . 8

2.2.2. Fuera del alcance . . . 8

2.2.3. Entradas externas . . . 8

2.2.4. Salidas externas . . . 8

2.2.5. Restricciones de la arquitectura . . . 9

3. Requisitos y atributos de calidad **9**

3.1. Requisitos de los Stakeholders . . . 9

3.1.1. Identificación de los interesados . . . 9

3.1.2. Cómo se interpreta el cuadro . . . 10

3.1.3. Criterios de éxito del PMP y su implicación arquitectónica . . . 11

3.2. Atributos de Calidad . . . 12

3.2.1. Marco de clasificación . . . 12

3.2.2. Rendimiento . . . 13

3.2.3. Confiabilidad . . . 14

3.2.4. Usabilidad . . . 14

3.2.5. Soporte . . . 15

3.2.6. Priorización y compromisos entre atributos . . . 15

3.3. Supuestos . . . 16

4. Principios y decisiones arquitectónicas **18**

4.1. Estilo base . . . 18

4.1.1. Orquestador del sistema . . . 18

4.1.2. Componentes del núcleo . . . 20

4.1.3. Arquitectura basada en plugins . . . 20

4.1.4. Principio de desacoplamiento y extensibilidad . . . 21
4.1.5. Justificación . . . 21
4.1.6. Decisiones arquitectónicas . . . 21
4.1.7. Patrones utilizados . . . 22

5. Vistas Arquitectónicas (4+1) 23
5.1. Vista de Escenarios . . . 23
5.1.1. Propósito . . . 23
5.1.2. Notación . . . 24
5.1.3. Visión general del modelo . . . 24
5.1.4. Subsistema Interfaz y Usabilidad . . . 25
5.1.5. Subsistema Núcleo y Ciclo de Vida . . . 26
5.1.6. Subsistema Preprocesamiento . . . 28
5.1.7. Subsistema Análisis . . . 29
5.1.8. Subsistema Exportación . . . 30
5.2. Vista Lógica . . . 30
5.2.1. Propósito . . . 30
5.2.2. Notación . . . 31
5.2.3. Visión general del modelo . . . 31
5.2.4. Elementos del núcleo . . . 31
5.2.5. Servicios del sistema . . . 32
5.2.6. Elementos de presentación . . . 33
5.2.7. Contrato de extensión . . . 34
5.2.8. Extensiones . . . 34
5.2.9. Modelo de datos . . . 35
5.2.10. Relaciones estructurales . . . 35
5.2.11. Justificación de las decisiones . . . 36
5.2.12. Trazabilidad con los requerimientos . . . 37
5.3. Vista de Procesos . . . 37
5.4. Vista Física . . . 39

6. Atributos de calidad 40
6.1. Escenarios de calidad . . . 40
6.1.1. Escenario de Calidad 1: Integración de un Plugin Externo . . . 40
6.1.2. Escenario de Calidad 2: Carga de Señal de Gran Tamaño . . . 40
6.1.3. Escenario de Calidad 3: Manejo de Errores en Tiempo de Ejecución . . . 41
6.1.4. Escenario de Calidad 4: Exportación de Datos o Gráficas . . . 41

6.1.5. Escenario de Calidad 5: Uso del Sistema sin Experiencia Técnica 42
6.1.6. Escenario de Calidad 6: Rendimiento con Múltiples Señales Cargadas . . . 42
6.1.7. Escenario de Calidad 7: Resiliencia ante Archivos Corruptos o Incompatibles 43
6.1.8. Escenario de Calidad 8: Consistencia entre Datos Ingresados y Gráficas Generadas 43
6.2. Reglas de compatibilidad y versionado 43
6.3. Métricas y plan de verificación 44
6.4. Plan de evidencia 45
6.5. Conclusión 46

7. Anexos **46**
7.1. Anexo A — Relación con el SRS 46
7.2. Anexo B — Relación con el SDD 46
7.3. Anexo C — Trazabilidad global de artefactos 46
7.4. Anexo D — Diagramas y tablas de referencia 46

8. Referencias **46**

Índice de figuras

1. Diagrama conceptual de la arquitectura microkernel de GammaLab 2.0, mostrando la relación entre el núcleo central, los servicios compartidos y los plugins especializados. 19

2. Diagrama de casos de uso del sistema completo Gamma Lab 2.0 (23 CU en 5 subsistemas). Las relaciones *include*/*extend* se tomaron literalmente del campo "Extensiones" de cada ficha; ninguna es inferida por similitud temática. 25

3. Diagrama de clases del sistema Gamma Lab 2.0. Los elementos en verde se encuentran implementados y los elementos en rojo están planificados. 31

4. Flujo maestro de sesión de Gamma Lab 2.0, con carriles Investigador/Sistema. Verde = ya existe, rojo = nuevo, blanco = mixto. 38

5. Vista física de Gamma Lab 2.0: estación única de escritorio y su superficie de E/S. 39

Índice de cuadros

1. Acrónimos y términos utilizados en el documento 6
2. Stakeholders de GammaLab v2.0, rol según el PMP y necesidades que condicionan la arquitectura 10
3. Matriz de interés de los stakeholders por atributo de calidad 11
4. Criterios de éxito del PMP con impacto arquitectónico 11
5. Resumen de los atributos de calidad de GammaLab v2.0 12
6. Compromisos entre atributos de calidad y decisión adoptada 15
7. Supuestos de GammaLab v2.0, su fuente y su impacto 16
8. ADRs resumidas: decisión, motivación y consecuencias. 22
9. Patrones arquitectónicos aplicados en GammaLab 2.0. 23
10. Casos de uso del subsistema Interfaz y Usabilidad 26
11. Casos de uso del subsistema Núcleo y Ciclo de Vida 27
12. Casos de uso del subsistema Preprocesamiento 28
13. Casos de uso del subsistema Análisis 29
14. Casos de uso del subsistema Exportación 30
15. Extensiones representadas en la vista lógica. 34
16. Relaciones estructurales de la vista lógica. 36
17. Trazabilidad entre elementos de la vista lógica y requerimientos. 37
18. Escenario de calidad 1 — Integración de un plugin externo 40
19. Escenario de calidad 2 — Carga de señal de gran tamaño 40
20. Escenario de calidad 3 — Manejo de errores en tiempo de ejecución 41
21. Escenario de calidad 4 — Exportación de datos o gráficas 41
22. Escenario de calidad 5 — Operación por un usuario sin experiencia técnica 42
23. Escenario de calidad 6 — Rendimiento con múltiples señales cargadas 42
24. Escenario de calidad 7 — Resiliencia ante archivos corruptos o incompatibles . . 43
25. Escenario de calidad 8 — Consistencia entre datos ingresados y gráficas generadas 43
26. Versiones de dependencias externas ancladas en `requirements.txt` 44
27. Métricas, método de verificación y criterio de aceptación por atributo de calidad . 45

1. Introducción

1.1. Propósito

El presente documento tiene como propósito definir y documentar la arquitectura de GammaLab v2.0, estableciendo los principios, decisiones y lineamientos que orientan su evolución respecto a GammaLab v1.0. Para ello, describe la estructura arquitectónica de la solución, sus componentes principales, las relaciones entre ellos y las decisiones que permiten soportar la incorporación de nuevas capacidades de análisis de señales EEG, mejorar el rendimiento del sistema y facilitar su evolución y extensibilidad. Asimismo, presenta las vistas arquitectónicas bajo el modelo 4+1 y establece los criterios necesarios para evaluar que la arquitectura propuesta responda a los atributos de calidad definidos para el sistema, manteniendo coherencia con los requerimientos y objetivos establecidos para GammaLab v2.0.

1.2. Alcance

Este documento abarca la definición de los principios y decisiones arquitectónicas que rigen la estructura de Gamma Lab v2.0. La arquitectura adopta un estilo Microkernel, compuesto por un núcleo de servicios esenciales y un conjunto de plugins que extienden las capacidades del sistema mediante componentes desacoplados. Asimismo, se describen las vistas arquitectónicas bajo el modelo 4+1, los componentes y sus relaciones, los atributos de calidad, las estrategias para alcanzarlos y las métricas utilizadas para su evaluación. Finalmente, se documentan los principales riesgos y decisiones arquitectónicas y su trazabilidad con el SRS de Gamma Lab.

Fuera del alcance de este documento quedan los aspectos relacionados con la administración del proyecto, la implementación detallada de los componentes, los procedimientos de pruebas y las instrucciones de uso del sistema, los cuales se encuentran documentados en los artefactos correspondientes.

1.3. Definiciones y acrónimos

Cuadro 1: Acrónimos y términos utilizados en el documento

Sigla / Término	Definición
SAD	Software Architecture Document. Documento de arquitectura del sistema.
SRS	Software Requirements Specification. Documento de especificación de requisitos.
SDD	Software Design Document. Documento de diseño técnico.
IoC	Inversión de Control. Principio que delega el flujo de ejecución a un contenedor.
DI	Dependency Injection. Técnica para resolver dependencias desde un contenedor externo.
MVP / MVC	Patrones de separación de interfaz y lógica (Model–View–Presenter / Model–View–Controller).
Template Method	Patrón que define un flujo general con pasos sobrescribibles.
Adapter	Patrón para estandarizar interfaces incompatibles.
Caching	Estrategia para evitar recalcular o recomponer estructuras de datos.

Sigla / Término	Definición
Python	Lenguaje de programación principal de *Gamma Lab*.
PyQt5	Framework de interfaz gráfica basado en Qt para Python.
VTK	Visualization Toolkit. Biblioteca para visualización científica 2D/3D.
QVTK	Interfaz `QVTKRenderWindowInteractor` para incrustar VTK en aplicaciones Qt.
NumPy	Librería para cálculo numérico vectorizado.
EDF	European Data Format. Formato estándar para señales biomédicas.
ABF	Axon Binary File. Formato de archivo utilizado por Axon Instruments para registros biomédicos.
GUI Toolkit	Conjunto de herramientas para construir interfaces gráficas (Qt, PyQt, VTK).

1.4. Audiencia

Este documento está dirigido al equipo de desarrollo, así como a cualquier stakeholder con formación técnica que requiera comprender las decisiones arquitectónicas del sistema. Asimismo, está orientado a todos los investigadores y personas del Laboratorio de Electrofisiología de la Universidad de Colombia que estén interesados en conocer la estructura interna, los principios de diseño y los mecanismos de GammaLab v2.0 para el uso, mantenimiento y futuros cambios.

1.5. Referencias relacionadas

Este SAD se alinea con:

- **SRS de Gamma Lab**: requisitos funcionales y no funcionales del sistema.

Las referencias técnicas externas (PyQt, VTK, NumPy, EDF/ABF, YAML).

2. Contexto del sistema

2.1. Visión general

GammaLab v2.0 está estructurado como una aplicación de escritorio tipo standalone diseñada para ser ejecutada de forma local, sin requerir conectividad de red. Su arquitectura se basa en un patrón modular de microkernel, donde un núcleo central asíncrono (desarrollado en Python con el framework gráfico PySide6 y la librería de visualización científica VTK) orquesta la ejecución y comunicación de plugins independientes a través de un almacén de datos en memoria (DataStore). La evolución fundamental de esta segunda iteración se muestra una evolución de los hilos únicos de Python, mediante la incorporación de estrategias de paralelización basadas en multiprocesamiento y multithreading en el kernel. Esto permite que el sistema distribuya las tareas pesadas (como la transformada Wavelet y el acoplamiento cruzado de frecuencias) sin congelar la interfaz de usuario.

2.2. Límites del sistema

2.2.1. Dentro del alcance

- **Ingreso y lectura:** Carga nativa de registros electroencefalográficos (EEG) almacenados localmente en formatos estándares .edf, .abf, .bdf y .mat.
- **Procesamiento y segmentación:** Aplicación de filtros digitales de corte, remoción de artefactos de estimulación y segmentación de señales en ensayos (trials) mediante umbrales en mV u operaciones de intervalo interestímulo (ISI).
- **Procesamiento espectral y tiempo-frecuencia:** Cálculo de promedios de Potenciales Relacionados a Eventos (ERP) con visualización de Butterfly plot bajo demanda, análisis de Transformada Rápida de Fourier (FFT), Densidad Espectral de Potencia (PSD) y Transformada Wavelet Continua con normalización y escalas personalizables.
- **Análisis de conectividad y acoplamiento avanzado:** Ejecución del análisis de acoplamiento fase-amplitud (PAC) tanto de un canal como cruzado intercanal.
- **Persistencia del espacio de trabajo:** Guardado y restauración del estado de la sesión analítica mediante archivos de proyecto local con extensión personalizada .glab.
- **Exportación de resultados:** Generación de archivos de datos numéricos estructurados (CSV con ejes X simplificados, Excel, JSON) y representaciones gráficas de alta resolución, incluyendo formato de imagen estándar (PNG, JPG, TIFF) e imagen vectorial científica (EMF).

2.2.2. Fuera del alcance

- La comunicación o adquisición de señales EEG en tiempo real mediante conexiones directas a equipos de amplificación, electrodos o diademas inalámbricas.
- La persistencia de proyectos en bases de datos centralizadas, servicios en la nube, servidores remotos o API web distribuidas; toda la persistencia es local e individual.
- El soporte operativo para plataformas web, sistemas móviles o sistemas operativos Unix/macOS en distribución comercial nativa (el instalador standalone se restringe a Windows de 64 bits).
- Mecanismos de control de acceso multiusuario, autenticación de seguridad en red, cifrado de datos clínicos o sincronización colaborativa síncrona.

2.2.3. Entradas externas

- Señales neurofisiológicas digitalizadas en formatos binarios compatibles (.edf, .abf, .bdf, .mat).
- Configuración alfanumérica de entrada por el usuario clínico (frecuencias de muestreo, umbrales de trial en mV, bandas de frecuencia de fase/amplitud para PAC/PLV y ciclos de wavelet).
- Eventos de interacción directa del investigador con la GUI mediante teclado y ratón para realizar zoom multidireccional, desplazamiento temporal (pan) y selección de puntos específicos sobre las gráficas para la toma de medidas.

2.2.4. Salidas externas

- Gráficas 2D renderizadas mediante VTK embebido en PySide6, tales como comodulogramas de PAC, mapas de calor de coherencia temporal-frecuencial y espectrogramas Wavelet.

- Archivos con metadatos y estados serializados .glab que contienen los análisis intermedios, rutas absolutas y configuraciones del proyecto.
- Tablas de resultados analíticos en formato Excel (.xlsx), JSON y archivos CSV optimizados para evitar redundancia de ejes temporales.
- Archivos de imagen estándar (PNG/JPG/TIFF) y gráficos vectoriales de alta precisión (EMF) para uso directo en publicaciones indexadas.

2.2.5. Restricciones de la arquitectura

- **Operación standalone y offline:** El sistema debe ejecutarse de forma local en la máquina del investigador de forma estricta. No se permite el uso de servicios en la nube, APIs de procesamiento externas ni el uso de bases de datos distribuidas.
- **Entorno tecnológico base:** El núcleo matemático y la lógica de interacción deben desarrollarse bajo el lenguaje Python (versión mínima 3.11), utilizando Pyside6 para la interfaz y VTK para la visualización científica.
- **Compatibilidad de datos:** El módulo de entrada/salida soporta y procesa archivos de señales en formatos como: EDF, ABF, BDF, MAT y GLAB.

3. Requisitos y atributos de calidad

Una arquitectura no se justifica por sí misma, sino por las necesidades que debe satisfacer. Este capítulo establece ese punto de partida: identifica a las personas y entidades con interés en GammaLab v2.0 y traduce sus expectativas en requisitos que condicionan la estructura del sistema (Sección 3.1), define las cualidades que la solución debe exhibir y cómo se miden (Sección 3.2) y declara las condiciones que se dan por ciertas durante el diseño (Sección 3.3).

Los stakeholders y sus roles provienen del Plan de Administración del Proyecto (PMP), mientras que los requisitos no funcionales provienen de la Especificación de Requisitos de Software (SRS). Así, cada decisión descrita en los capítulos siguientes puede rastrearse hasta una necesidad concreta de un interesado.

3.1. Requisitos de los Stakeholders

3.1.1. Identificación de los interesados

Un *stakeholder* es toda persona o entidad que afecta el desarrollo de GammaLab v2.0 o se ve afectada por él. La identificación que se presenta a continuación es coherente con la estructura organizacional y la distribución de roles definidas en el PMP: el Laboratorio de Electrofisiología de la Universidad Nacional de Colombia actúa como cliente, usuario final y validador; la directora del proyecto actúa como *Product Owner* y puente entre el cliente y el equipo; y el equipo de desarrollo, conformado por cuatro estudiantes con un integrante que asume funciones equivalentes a *Scrum Master*, ejecuta el trabajo técnico. A ellos se suman los revisores académicos del trabajo de grado, que evalúan el producto final, y la comunidad científica, beneficiaria indirecta por la naturaleza de código abierto del sistema (licencia MIT).

Cuadro 2: Stakeholders de GammaLab v2.0, rol según el PMP y necesidades que condicionan la arquitectura

Stakeholder	Rol en el proyecto	Necesidades e intereses	Atributos de calidad asociados
Laboratorio de Electrofisiología (Universidad Nacional de Colombia)	Cliente, usuario final y validador. Participa en las Fases 1, 2 y 4.	Una herramienta gratuita e intuitiva que no exija conocimientos de programación; abrir sus registros (`.abf`, `.edf`, `.mat`) y ejecutar análisis de frecuencia, tiempo-frecuencia y acoplamiento fase-amplitud (PAC) sin bloquear la interfaz; resultados precisos y confiables; poder guardar y retomar su trabajo; y documentación clara.	Usabilidad, Rendimiento, Confiabilidad.
Directora del proyecto	*Product Owner*. Representa al cliente y prioriza el *Product Backlog*.	Rigor científico de los algoritmos de análisis, cumplimiento de los estándares de calidad y alineación del producto con los objetivos del proyecto; retroalimentación oportuna sobre diseño, código y documentación, y supervisión técnica y académica.	Confiabilidad, Soporte.
Equipo de desarrollo (cuatro *Scrum Developers*, uno con funciones de *Scrum Master*)	Diseña, implementa, integra, prueba y documenta el sistema.	Una arquitectura extensible (microkernel y plugins) que permita incorporar nuevos análisis sin modificar el núcleo; comprender y reutilizar el código heredado de la versión 1.0; procesamiento en segundo plano; y código verificable mediante pruebas automatizadas.	Soporte, Rendimiento.
Revisores académicos del trabajo de grado	Evalúan el cumplimiento de los entregables y de los objetivos.	Verificar el cumplimiento de los requisitos y de la metodología, y poder constatar con evidencia cuantitativa la mejora de GammaLab v2.0 respecto a la versión 1.0.	Rendimiento, Confiabilidad (verificabilidad).
Comunidad científica y usuarios de software libre	Beneficiario indirecto (derivado de la licencia MIT definida en el PMP).	Reutilizar, adaptar y extender el código abierto; reproducibilidad de los resultados y transparencia metodológica.	Soporte, Confiabilidad.

3.1.2. Cómo se interpreta el cuadro

La distinción entre el Laboratorio y la directora es deliberada. El Laboratorio es quien usa el sistema y valida los resultados; la directora, en su rol de *Product Owner*, canaliza y prioriza esas necesidades ante el equipo. Por ello, los requisitos del Laboratorio se tratan como **requisitos**

primarios, mientras que los de los demás interesados cumplen una función de gobierno (directora y revisores) o de sostenibilidad (equipo y comunidad). El equipo de desarrollo es, además, quien convive con las consecuencias de la arquitectura a largo plazo, razón por la cual su interés en la extensibilidad y la mantenibilidad pesa tanto como el del usuario final en la usabilidad.

La Tabla 3 resume qué atributos de calidad importan a cada interesado. Un círculo lleno (●) indica un interés principal, derivado directamente de sus necesidades; un círculo vacío (○) indica un interés secundario.

Cuadro 3: Matriz de interés de los stakeholders por atributo de calidad

Stakeholder	Usabilidad	Rendimiento	Confiabilidad	Soporte
Laboratorio de Electrofisiología	●	●	●	○
Directora del proyecto (*Product Owner*)	○	○	●	●
Equipo de desarrollo	○	●	○	●
Revisores académicos	○	●	●	○
Comunidad científica	○	—	○	●

3.1.3. Criterios de éxito del PMP y su implicación arquitectónica

El PMP define criterios de éxito cuantificables para el proyecto. No todos son de naturaleza arquitectónica (por ejemplo, la entrega en la fecha acordada o la completitud de la documentación son compromisos de gestión), pero cuatro de ellos imponen condiciones directas sobre el diseño del sistema y se recogen en la Tabla 4.

Cuadro 4: Criterios de éxito del PMP con impacto arquitectónico

Criterio de éxito (PMP)	Atributo	Implicación para la arquitectura
Mejora de al menos 20–30 % en los tiempos de procesamiento respecto a GammaLab v1.0, derivada de la paralelización.	Rendimiento	Las operaciones de cómputo deben ejecutarse fuera del hilo de la interfaz, a través de un servicio de tareas central (NFR-P-14, NFR-S-04), de modo que el trabajo pesado no compita con la interacción.
El módulo de acoplamiento fase-amplitud (PAC) opera sin errores críticos y produce resultados reproducibles y validables.	Confiabilidad	El análisis PAC se ejecuta como tarea cancelable (NFR-P-16) y el procesamiento no altera la señal ni los trials de origen (NFR-R-05), lo que permite repetir un análisis y obtener el mismo resultado.
Los problemas de la versión 1.0 priorizados con el cliente quedan corregidos y validados por él.	Confiabilidad, Soporte	La liberación de recursos al salir de un módulo (NFR-P-02), la terminación de hilos (NFR-R-01) y el acceso único al estado compartido (NFR-R-02) atacan la inestabilidad heredada, mientras que una vía pública para modificar los trials (NFR-S-02) evita que el problema reaparezca por acoplamiento.

Criterio de éxito (PMP)	Atributo	Implicación para la arquitectura
El usuario final confirma en la Fase 4 que el sistema es intuitivo, que los resultados son precisos y que las funcionalidades responden a sus necesidades.	Usabilidad, Confiabilidad	Se requiere retroalimentación visual del progreso, mensajes diferenciados por severidad, ayudas contextuales y resultados mostrados en la misma pantalla en que se solicitaron (NFR-U-01 a NFR-U-09).
Cobertura de pruebas unitarias de al menos 80 %.	Soporte	La lógica de negocio debe poder verificarse sin la interfaz gráfica: servicios compartidos con una única instancia inyectada desde el núcleo (NFR-S-03) y plugins que no acceden a estructuras internas de otros módulos (NFR-S-02).

3.2. Atributos de Calidad

3.2.1. Marco de clasificación

Los atributos de calidad describen *cómo* debe comportarse el sistema, a diferencia de los requisitos funcionales, que describen *qué* debe hacer. Son relevantes para este documento porque, de todos los requisitos, son los que más condicionan la arquitectura: la manera de ejecutar un cálculo pesado, de aislar una extensión defectuosa o de representar una señal de gran tamaño no se resuelve con código local, sino con decisiones estructurales.

La clasificación sigue el modelo FURPS+ adoptado por el SRS, donde cada requisito no funcional se identifica con el código **NFR-X-YY** (X es la categoría y YY el consecutivo). En esta iteración se especificaron 36 requisitos no funcionales distribuidos en cuatro categorías, que constituyen los atributos de calidad del sistema: Usabilidad (9), Rendimiento (17), Confiabilidad (5) y Soporte (5). La Tabla 5 los sintetiza.

Cuadro 5: Resumen de los atributos de calidad de GammaLab v2.0

Atributo	Qué garantiza	Medidas representativas	Requisitos
Rendimiento	Que el sistema responda con fluidez y use los recursos de forma controlada, incluso con señales grandes.	Carga de señal $\leq$ 2 s; Wavelet $\leq$ 10 s; interfaz con respuesta $\leq$ 0,3 s; $\geq$ 24 FPS con 3 canales; cambio de módulo $\leq$ 500 ms; PAC de 20×20 bandas $\leq$ 30 s; arranque $\leq$ 8 s.	NFR-P-01 a NFR-P-17
Confiabilidad	Que el sistema sea estable en sesiones prolongadas y que los datos del investigador no se corrompan.	Terminación garantizada de hilos; un único escritor por dato; señal de origen inmutable; representaciones gráficas invalidadas al cambiar los datos.	NFR-R-01 a NFR-R-05

Atributo	Qué garantiza	Medidas representativas	Requisitos
Usabilidad	Que un investigador sin formación en programación pueda operar el sistema sin ayuda.	Retroalimentación de progreso en toda operación de más de 1 s; mensajes por severidad; ayuda emergente en cada control; pantalla de bienvenida e inicio.	NFR-U-01 a NFR-U-09
Soporte	Que el sistema se pueda instalar, extender y mantener sin degradar lo existente.	Plugin nuevo registrable sin modificar el núcleo; vía pública única para los trials; servicios compartidos con instancia única; ejecutable instalable.	NFR-S-01 a NFR-S-05

Cada atributo se desarrolla a continuación en cuatro partes: por qué importa, qué se exige, cómo lo resuelve la arquitectura y cómo se verifica. Los escenarios detallados (estímulo, respuesta y medida) se presentan en la Sección 6, y las métricas con su método de verificación en la Tabla 27.

3.2.2. Rendimiento

Por qué importa. Las señales neurofisiológicas son densas y los análisis de frecuencia y tiempo-frecuencia (Wavelet, PAC) son computacionalmente costosos. Si el cálculo se ejecuta en el mismo hilo que la interfaz, la aplicación deja de responder y el investigador no puede distinguir un cálculo largo de una caída. El rendimiento es, además, el atributo con el que los revisores académicos medirán la mejora frente a la versión 1.0.

Qué se exige. Los requisitos fijan umbrales verificables sobre el hardware de referencia del laboratorio. Entre los más condicionantes se encuentran: cargar una señal en $\leq$ 2 s (NFR-P-03), completar el Wavelet en $\leq$ 10 s (NFR-P-04), sostener al menos 24 FPS al interactuar con tres canales (NFR-P-05), mantener la interfaz responsiva en $\leq$ 0,3 s mientras corre un análisis en segundo plano (NFR-P-10) y completar un comodulograma PAC de hasta 20×20 pares de bandas en $\leq$ 30 s con posibilidad de cancelarlo (NFR-P-16). El consumo de memoria también está acotado: la memoria de una señal debe liberarse al cerrar el proyecto o al cargar otra (NFR-P-01, NFR-P-08) y los recursos gráficos deben liberarse al salir de cada módulo (NFR-P-02).

Cómo lo resuelve la arquitectura.

- **Ejecución fuera del hilo de la interfaz.** Toda operación de cómputo de los plugins se despacha a través de un servicio de tareas registrado en el núcleo, que gestiona la cola, la cancelación y la notificación de progreso (NFR-P-14, NFR-S-04).
- **Submuestreo solo visual.** Para conservar la fluidez al graficar, se limita a 2 000 puntos por serie y 200 curvas simultáneas, y se reduce únicamente la representación, nunca los datos (NFR-P-15).
- **Conversión de datos en bloque.** Las representaciones gráficas se construyen con operaciones vectorizadas y no recorriendo los datos elemento por elemento (NFR-P-13).
- **Reutilización de la representación.** La tabla gráfica de una señal se conserva mientras los datos no cambien, evitando reconstruirla en cada dibujo (NFR-R-04).

Cómo se verifica. Escenarios de Calidad 2 (carga de señal de gran tamaño) y 6 (múltiples señales cargadas), y las métricas de la Tabla 27.

3.2.3. Confiabilidad

Por qué importa. Un análisis científico solo es útil si sus resultados son correctos y reproducibles, y una sesión de trabajo larga no debe perderse por el fallo de un componente. Esta preocupación proviene tanto del Laboratorio, que debe poder confiar en los resultados, como de la directora, responsable de validar el rigor del producto. Además, el PMP identifica la estabilidad técnica como una de las limitaciones principales de la versión 1.0 que esta iteración debe corregir.

Qué se exige. El sistema debe garantizar la terminación y liberación de todo hilo trabajador cuando se cancela una tarea o se detiene el plugin que la originó (NFR-R-01); sincronizar el acceso al estado compartido de la sesión de modo que un solo propietario escriba cada dato (NFR-R-02); retirar los detectores de eventos de las gráficas al salir de cada módulo (NFR-R-03); descartar de forma explícita las representaciones gráficas cuando los datos cambian, para no mostrar una versión obsoleta (NFR-R-04); y no modificar jamás la señal cargada ni los trials originales (NFR-R-05).

Cómo lo resuelve la arquitectura.

- **Propiedad única de los datos.** El *DataStore* centraliza el estado de la sesión y se accede a él con un único escritor, de modo que la concurrencia no produzca estados inconsistentes (NFR-R-02).
- **Datos de origen inmutables.** Todo preprocesamiento produce un conjunto derivado y conserva disponible el original, de modo que cualquier análisis puede repetirse desde la fuente (NFR-R-05).
- **Ciclo de vida explícito de los plugins.** Cada plugin libera sus recursos gráficos, observadores y tareas al detenerse, en lugar de limitarse a ocultar su interfaz (NFR-R-01, NFR-R-03, NFR-P-02).
- **Aislamiento de fallos.** Los errores de un plugin o de un archivo incompatible se capturan y se informan sin interrumpir la sesión.

Cómo se verifica. Escenarios de Calidad 3 (errores en tiempo de ejecución), 4 (exportación), 7 (archivos corruptos o incompatibles) y 8 (consistencia entre datos y gráficas).

3.2.4. Usabilidad

Por qué importa. El diferenciador del producto, según el PMP, es ser la única herramienta gratuita con interfaz gráfica intuitiva que no requiere programar. Si el investigador necesita asistencia para operar el sistema, ese diferenciador se pierde. La usabilidad es, por tanto, el criterio con el que el Laboratorio validará el producto en la Fase 4.

Qué se exige. Toda operación que tarde más de 1 s debe mostrar progreso (NFR-U-03); los mensajes deben diferenciar información, advertencia y error (NFR-U-01); el resultado debe mostrarse en la misma pantalla en que se solicitó (NFR-U-02); cada control relevante debe tener una descripción emergente (NFR-U-07) e icono (NFR-U-04); las funcionalidades de edición deben concentrarse en *Home* (NFR-U-08); y el sistema debe dar la bienvenida a usuarios nuevos y mostrar una pantalla de inicio durante la carga (NFR-U-05, NFR-U-09).

Cómo lo resuelve la arquitectura.

- **Separación entre interfaz y cómputo.** Como el cálculo corre fuera del hilo de la interfaz, esta puede mostrar progreso y seguir respondiendo (NFR-U-03 se apoya en NFR-P-14).
- **Interfaz uniforme entre plugins.** Todos los módulos comparten el mismo patrón de panel de parámetros, de modo que aprender uno facilita usar los demás.
- **Retroalimentación centralizada.** La notificación de mensajes sigue un mecanismo co-

mún que distingue su severidad (NFR-U-01).

Cómo se verifica. Escenario de Calidad 5 (uso del sistema sin experiencia técnica) y las pruebas de usabilidad con el Laboratorio previstas en la Fase 4.

3.2.5. Soporte

Por qué importa. GammaLab v2.0 nace como una evolución de la versión 1.0 y se concibe para seguir creciendo: nuevos análisis, nuevos formatos y nuevas necesidades de otros laboratorios. El Soporte agrupa lo necesario para que ese crecimiento no degrade lo ya construido, y es el atributo que más interesa al equipo de desarrollo y a la comunidad que reutilice el código.

Qué se exige. Un nuevo plugin de análisis debe poder incorporarse registrándolo en el núcleo, sin modificar el núcleo ni los plugins existentes (NFR-S-05); ningún módulo debe alcanzar la estructura interna de los datos, sino usar una vía pública única para leer, modificar e invalidar los trials (NFR-S-02); los servicios compartidos de exportación, medición y ajustes deben ser entregados por el núcleo con una única instancia (NFR-S-03); la ejecución pesada debe pasar por un servicio de tareas (NFR-S-04); y el sistema debe poder instalarse con un ejecutable que registre la extensión de proyecto `.glab` (NFR-S-01).

Cómo lo resuelve la arquitectura.

- **Estilo microkernel.** Un núcleo mínimo de servicios y extensiones desacopladas descubiertas de forma dinámica, de modo que agregar capacidades no obliga a recompilar ni a tocar el núcleo.
- **Inversión de control.** Los servicios comunes se registran en el núcleo y se entregan a los plugins, en lugar de que cada plugin los construya o importe directamente.
- **Encapsulamiento del modelo de datos.** La entidad que representa la señal expone una interfaz pública para operar sobre sus trials y su frecuencia de muestreo, de modo que un cambio interno no rompe a los módulos que dependen de ella.

Cómo se verifica. Escenario de Calidad 1 (integración de un plugin externo).

3.2.6. Priorización y compromisos entre atributos

Los atributos no son independientes: mejorar uno suele costar algo de otro. Hacer explícitos estos compromisos evita decisiones accidentales y permite justificarlas ante los interesados. La Tabla 6 recoge los más relevantes y la decisión adoptada en cada caso.

Cuadro 6: Compromisos entre atributos de calidad y decisión adoptada

Atributos en tensión	**Tensión**	**Decisión de la arquitectura**
Rendimiento frente a Confiabilidad	Submuestrear la gráfica acelera el dibujo, pero podría ocultar detalle de la señal.	El submuestreo se aplica únicamente a la representación visual y los datos permanecen intactos (NFR-P-15, NFR-R-05): se gana fluidez sin perder fidelidad en el análisis ni en la exportación.

Atributos en tensión	Tensión	Decisión de la arquitectura
Rendimiento frente a Confiabilidad	Ejecutar análisis en segundo plano introduce concurrencia y el riesgo de condiciones de carrera sobre los datos.	Se acepta la concurrencia a cambio de la fluidez, pero se compensa con un único escritor por dato (NFR-R-02) y con la terminación garantizada de los hilos (NFR-R-01).
Soporte frente a Rendimiento	Desacoplar los módulos mediante servicios y plugins añade capas de indirección.	Se prioriza el desacoplamiento, porque la indirección es pequeña frente al costo del cómputo científico; su efecto se vigila con las métricas de rendimiento (Tabla 27).
Usabilidad frente a Rendimiento	Informar el progreso exige comunicación constante entre el hilo de cómputo y la interfaz.	El servicio de tareas incorpora la notificación de progreso como parte de su contrato (NFR-S-04), de modo que ningún plugin la implemente por su cuenta.

3.3. Supuestos

Un supuesto es una condición que se considera cierta durante el diseño sin que el sistema pueda garantizarla. Documentarlos es importante porque, si alguno deja de cumplirse, las decisiones que dependen de él deben revisarse. Por eso, la Tabla 7 registra, para cada supuesto, su fuente y la consecuencia de que no se cumpla. Las restricciones que limitan el diseño, a diferencia de los supuestos, se establecen en la Sección 2.2 (*Límites del sistema*).

Cuadro 7: Supuestos de GammaLab v2.0, su fuente y su impacto

ID	Supuesto	Fuente	Impacto si no se cumple
Entorno de ejecución			
A-01	El sistema se ejecuta sobre Windows 10 o superior, en arquitectura de 64 bits.	SRS	No se garantiza el instalador ni la portabilidad prevista (NFR-S-01).
A-02	El equipo cuenta con arquitectura multinúcleo (más de cuatro núcleos) para el procesamiento en segundo plano.	Equipo	La mejora de rendimiento esperada por la paralelización (20–30 %) puede no alcanzarse.
A-03	El equipo dispone de recursos suficientes para manipular señales de gran tamaño, y los umbrales de rendimiento se miden sobre el hardware de referencia del laboratorio.	SRS, PMP	Los tiempos de carga, de graficación y de arranque (NFR-P-03, P-05, P-17) pueden superarse.
A-04	El usuario tiene permisos de lectura y escritura en el sistema de archivos local.	SRS	No es posible cargar señales, guardar proyectos ni exportar resultados.

ID	Supuesto	Fuente	Impacto si no se cumple
A-05	Las dependencias científicas requeridas (por ejemplo, NumPy, SciPy, PyWavelets, PyABF y PyEDFlib) están disponibles en el entorno de ejecución, ya sea instaladas o empaquetadas en el ejecutable.	SRS	Los módulos de lectura de archivos y de análisis no pueden ejecutarse.
Datos de entrada			
A-06	Las señales a procesar ya fueron adquiridas: el sistema se dedica al posprocesamiento y no a la adquisición en tiempo real.	Sección ef-sec:limites	Requeriría un subsistema de adquisición y comunicación con hardware, hoy fuera de la arquitectura.
A-07	Los archivos de señal están en un formato soportado (`.abf`, `.edf`, `.bdf` o `.mat`).	SRS	El archivo se rechaza y se informa al usuario (Escenario de Calidad 7).
A-08	Los archivos `.mat` no incluyen la frecuencia de muestreo, por lo que el investigador la conoce y la ingresa manualmente antes de ejecutar análisis de frecuencia.	R80, CU-018	Los análisis de frecuencia se calcularían sobre un eje temporal incorrecto y producirían resultados erróneos.
A-09	Las señales usadas en PAC tienen al menos un trial válido; el PAC inter-canal y el PPC requieren al menos dos canales con trials generados.	SRS	El análisis no puede ejecutarse o carece de significado.
A-10	Las bandas de frecuencia seleccionadas están por debajo de la frecuencia de Nyquist de la señal.	SRS	El cálculo espectral no es válido para las bandas que la superen.
Usuarios y participación de los interesados			
A-11	El usuario tiene conocimientos básicos o intermedios de análisis de señales EEG (dominios del tiempo, frecuencia y tiempo-frecuencia), sabe interpretar gráficas y comodulogramas, y maneja inglés técnico básico.	SRS	La interfaz y la terminología deberían simplificarse y complementarse con mayor ayuda contextual (Escenario de Calidad 5).
A-12	Los investigadores del Laboratorio participan activamente en las pruebas, la validación y la retroalimentación, y la institución facilita señales EEG para validar el sistema.	PMP	La validación de los resultados y del criterio de éxito de usuario final (Fase 4) quedaría sin sustento.
Arquitectura y extensiones			
A-13	La arquitectura microkernel heredada de la versión 1.0 es adecuada y escalable para incorporar las nuevas funcionalidades.	PMP	Sería necesario rediseñar el núcleo, con impacto en el alcance y el cronograma.

ID	Supuesto	Fuente	Impacto si no se cumple
A-14	Los módulos desarrollados por terceros respetan la arquitectura modular y las interfaces definidas por GammaLab.	SRS	Un plugin defectuoso podría comprometer la estabilidad de la sesión (NFR-R-01, NFR-S-05).

4. Principios y decisiones arquitectónicas

4.1. Estilo base

La arquitectura de GammaLab 2.0 adopta un estilo microkernel, en el que el sistema central o *core system* proporciona los servicios esenciales de comunicación, gestión de plugins, registro de eventos, coordinación de tareas y manejo de datos. Este núcleo actúa como mediador entre los distintos módulos funcionales del sistema, permitiendo que cada plugin se conecte e interactúe mediante una interfaz común, sin acoplarse directamente a la lógica interna del sistema.

El *Kernel* es el componente central de esta arquitectura y desempeña la función de orquestador principal. Su responsabilidad consiste en coordinar la ejecución de tareas, gestionar la cola de trabajos, supervisar el estado de cada operación, controlar la cancelación de procesos en ejecución y administrar la sincronización entre la interfaz gráfica y los módulos de análisis. De esta forma, la capa de presentación permanece responsiva mientras el procesamiento de señales y cálculos estadísticos se ejecutan en segundo plano.

La separación entre la lógica estable del sistema y la funcionalidad específica de análisis es una de las decisiones más relevantes del diseño. El *core* encapsula los servicios transversales de la aplicación, como el almacenamiento de datos, la gestión de eventos, la carga de archivos, la exportación de resultados y la configuración del entorno operativo. Por su parte, los plugins especializados implementan las distintas capacidades del sistema, como FFT, PSD, Wavelet, ERP, PAC y otras herramientas de análisis.

Esta estructura favorece la extensibilidad del sistema, ya que permite incorporar nuevos módulos funcionales sin modificar la lógica del núcleo. Además, reduce el acoplamiento entre componentes y mejora la mantenibilidad del proyecto a lo largo de su evolución. La relación entre el *core* y los plugins se establece a través de una API de integración basada en la interfaz *IPlugin*, la cual define el contrato que cada módulo debe cumplir para incorporarse al ecosistema de GammaLab 2.0.

En este sentido, la arquitectura del sistema se organiza en tres capas principales: la capa de presentación, donde se encuentra la interfaz gráfica desarrollada con PySide6 y Qt; la capa de coordinación, representada por el Kernel y los servicios compartidos del core; y la capa funcional, conformada por los plugins que implementan los distintos análisis y visualizaciones.

4.1.1. Orquestador del sistema

El Kernel de GammaLab 2.0 se define como el orquestador central del sistema. Su principal función es coordinar la ejecución de tareas complejas y garantizar que las operaciones intensivas de cómputo no bloqueen la interfaz gráfica. Esta decisión es clave para el rendimiento del sistema, ya que el análisis de señales EEG implica el manejo de datos de alta resolución, filtros, transformadas, estimaciones de potencia y análisis de acoplamiento cruzado de frecuencias.

Entre las funciones del orquestador se encuentran:

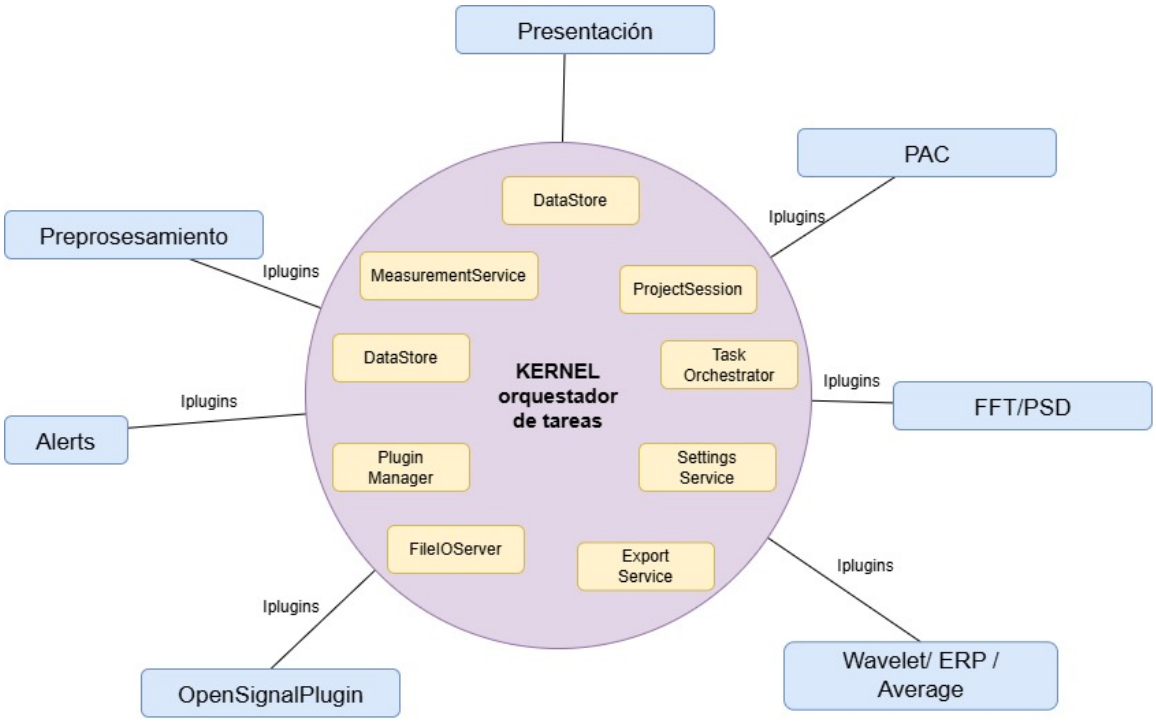


Figura 1: Diagrama conceptual de la arquitectura microkernel de GammaLab 2.0, mostrando la relación entre el núcleo central, los servicios compartidos y los plugins especializados.

- Registro y descubrimiento de plugins.
- Gestión de la cola de tareas y priorización de ejecución.
- Distribución de tareas entre hilos o procesos según su complejidad.
- Control del estado de cada operación: pendiente, ejecutándose, completada o cancelada.
- Supervisión del progreso de los análisis.
- Coordinación con los servicios del core.
- Entrega de resultados hacia la capa visual.
- Gestión de errores y alertas a la interfaz de usuario.

El orquestador actúa como el punto de integración entre la capa de presentación, los servicios del sistema y los módulos especializados. Cuando el usuario solicita ejecutar un análisis, la interfaz no invoca directamente la lógica del plugin; en cambio, la solicitud se envía al Kernel, quien valida el contexto, identifica el módulo apropiado, crea la tarea y la ejecuta de forma controlada. Esta estructura evita duplicación de lógica, reduce fallos de sincronización y mejora la trazabilidad de las operaciones.

En términos arquitectónicos, el Kernel también centraliza la administración de recursos del sistema. Esto incluye la gestión de la memoria compartida, la preparación de datos para análisis y la actualización del estado global del entorno. En un sistema orientado a señales neurofisiológicas, donde los volúmenes de información pueden ser elevados, esta coordinación es crítica para asegurar estabilidad, eficiencia y capacidad de respuesta.

4.1.2. Componentes del núcleo

El núcleo de GammaLab 2.0 está conformado por una serie de componentes que permiten la operación global del sistema. Estos módulos no son reemplazables por plugins, sino que constituyen la base funcional sobre la cual todo el entorno opera.

Los principales componentes del core son:

- **Kernel**: responsable de la orquestación y coordinación de tareas.
- **PluginManager**: encargado de registrar, validar y cargar los plugins disponibles.
- **DataStore**: almacena señales, resultados intermedios, metadatos, configuraciones y sesión activa.
- **FileIOService**: gestiona la lectura y validación de archivos EEG en formatos como ABF, EDF, BDF y MAT.
- **MeasurementService**: administra métricas, cálculos y resultados derivados del análisis.
- **SettingsService**: maneja la configuración general del sistema y preferencias del usuario.
- **ExportService**: se encarga de la exportación de resultados en formatos compatibles con análisis e informes.
- **ProjectSession**: gestiona la persistencia del espacio de trabajo mediante archivos de proyecto `.glab`, guardando y restaurando la señal activa, los trials, los análisis y los parámetros de la sesión.

El diseño del núcleo sigue un principio de servicio compartido, donde los plugins no crean instancias aisladas de cada funcionalidad, sino que acceden a los servicios del sistema mediante interfaces controladas por el Kernel. Este enfoque reduce el acoplamiento entre módulos y facilita la evolución del sistema sin afectar la lógica base.

4.1.3. Arquitectura basada en plugins

GammaLab 2.0 implementa una arquitectura modular basada en plugins, con el objetivo de ampliar la funcionalidad del sistema sin modificar el núcleo central. Cada plugin encapsula una capacidad específica de análisis o visualización, y se integra con la aplicación mediante un contrato común definido por la interfaz *IPlugin*.

Un plugin de GammaLab 2.0 puede contener:

- lógica de procesamiento,
- componentes de interfaz gráfica,
- configuraciones específicas del análisis,
- metadatos de identificación,
- conexión con el DataStore y el Kernel.

El *PluginManager* realiza el descubrimiento dinámico de nuevos módulos, validando la presencia de metadatos y registrando cada extensión dentro del ecosistema del sistema. Esta estructura permite que el sistema evolucione de forma incremental: cada nuevo módulo puede añadirse a la aplicación sin recompilar el núcleo ni modificar la lógica general del software.

4.1.4. Principio de desacoplamiento y extensibilidad

La arquitectura de GammaLab 2.0 se fundamenta en el principio de desacoplamiento funcional. La capa de presentación, el Kernel y los plugins mantienen una separación clara de responsabilidades. Esta decisión permite que cada módulo evolucione de forma independiente sin comprometer la estabilidad general del sistema. Por ejemplo, una modificación en la interfaz de usuario no exige cambiar la lógica del análisis; un nuevo módulo de procesamiento puede agregarse sin alterar la estructura del núcleo; y una actualización en la gestión de archivos no afecta directamente a la visualización de resultados.

La extensibilidad también se ve reforzada por un conjunto de mecanismos bien definidos:

- Registro dinámico de plugins.
- Contrato de integración mediante IPlugin.
- Gestión centralizada de datos mediante DataStore.
- Coordinación de tareas mediante el Kernel.
- Desacoplamiento entre lógica funcional y visualización.

4.1.5. Justificación

La arquitectura adoptada para GammaLab 2.0 corresponde a un estilo microkernel, en el que el núcleo central del sistema es el core y, dentro de este, el Kernel cumple la función de orquestador principal. Esta decisión permite separar la lógica estable del sistema de las funcionalidades específicas de análisis, manteniendo un diseño modular, extensible y de bajo acoplamiento.

El Kernel centraliza la coordinación de tareas, la gestión de la cola de ejecución, la supervisión del estado de cada operación y la delegación de la lógica funcional a los plugins. Esta decisión es necesaria porque el procesamiento de señales EEG requiere operaciones computacionalmente intensivas, como PAC, FFT, PSD y Wavelet, que no deben ejecutarse directamente en la interfaz gráfica. Además, al separar la lógica de coordinación del procesamiento funcional, el sistema mantiene un mejor control del flujo de trabajo, facilita la gestión de errores y cancelaciones, y permite una ejecución más ordenada de tareas en segundo plano. Esto mejora la experiencia del usuario, reduce la carga operativa del sistema y favorece la extensibilidad del proyecto, ya que nuevos plugins pueden añadirse sin afectar el funcionamiento del núcleo ni la interfaz.

Esta organización facilita la incorporación de nuevos módulos sin modificar el núcleo, reduce la dependencia entre componentes y permite un mejor control del flujo de datos y de los resultados. La integración con servicios compartidos como DataStore, FileIOService, MeasurementService y ExportService garantiza un manejo más ordenado de la información, la persistencia y la visualización de resultados.

4.1.6. Decisiones arquitectónicas

Conforme evolucionaba el proyecto, también lo hacían las decisiones arquitectónicas necesarias para alcanzar el estado final de la implementación. Los problemas, restricciones y hallazgos identificados durante el proceso guiaron la selección y el ajuste de cada componente del diseño. A continuación se listan las decisiones arquitectónicas con mayor impacto dentro del proyecto:

Cuadro 8: ADRs resumidas: decisión, motivación y consecuencias.

ID	Decisión	Motivación	Consecuencias
ADR-01	Estilo arquitectónico de microkernel.	Permitir la incorporación de nuevas funcionalidades al sistema, incluso en tiempo de ejecución.	Sistema modular, ampliable y con bajo acoplamiento; mayor complejidad inicial.
ADR-02	Separación entre core, plugins y app.	Organizar responsabilidades y mantener claridad modular.	Facilita la comprensión, el mantenimiento y el desarrollo incremental.
ADR-03	Interfaz IPlugin.	Aplicar inversión de control para que los plugins definan únicamente su lógica.	Menor acoplamiento.
ADR-04	Carga dinámica de plugins.	Permitir la incorporación de módulos sin recompilar.	Mejor extensibilidad y colaboración entre equipos.
ADR-05	Uso de Python, PySide6 y VTK.	Utilizar librerías maduras para visualización científica y GUI.	Base sólida para análisis; requiere control cuidadoso de rendimiento y memoria.
ADR-06	Uso de DataStore.	Evitar duplicaciones y permitir la propagación reactiva de cambios.	Coherencia global y ahorro de memoria; requiere un diseño cuidadoso.
ADR-07	Kernel como orquestador central.	Ejecutar tareas pesadas sin bloquear la interfaz ni acoplar cada módulo al core.	Mayor control de concurrencia, cancelación, prioridad y trazabilidad.
ADR-08	Estructura estándar del plugin.	Unificar lógica, interfaz gráfica, metadatos y configuración.	Mejor mantenibilidad y estandarización.
ADR-09	Soporte explícito para PAC y análisis de acoplamiento.	Incorporar análisis avanzados de señales neurofisiológicas.	Mayor alcance funcional y nuevas dependencias especializadas.

4.1.7. Patrones utilizados

A continuación se presenta un resumen de los patrones arquitectónicos aplicados en el diseño de GammaLab 2.0, alineados con el estilo microkernel, el mecanismo de plugins y los servicios internos del sistema:

Cuadro 9: Patrones arquitectónicos aplicados en GammaLab 2.0.

Patrón	**Problema que resuelve**	**Aplicación en GammaLab 2.0**
Plugin	Agregar capacidades sin modificar el núcleo del sistema.	Cada plugin define un archivo `properties.yaml` y una clase que implementa `IPlugin`. El Kernel los descubre dinámicamente y los vincula en tiempo de ejecución.
IoC / Service Locator (Kernel)	Reducir el acoplamiento y centralizar los servicios compartidos.	El Kernel registra y expone servicios como *DataStore*, *FileIOService*, *ExportService*, *MeasurementService* y *SettingsService*. Los plugins acceden a estos servicios sin crear instancias propias.
Adapter (NumPy a VTK)	Integrar estructuras científicas con componentes de visualización.	El módulo *vtk adapters* convierte arrays de NumPy en *vtkTable* o estructuras compatibles para su uso en *vtkChartXY* y otros componentes VTK.
MVP / MVC en interfaz	Separar la vista de la lógica de interacción.	*MainWindow* gestiona la navegación y composición; cada plugin proporciona su interfaz mediante `get_widget()`, mientras que la lógica reside en `process()`, `start()` y `stop()`.
Caching (datos y widgets)	Reducir la latencia y la recomputación innecesaria.	Se almacenan en caché datos, resultados intermedios y widgets de plugins previamente inicializados.
Template Method (ciclo de vida)	Definir puntos de extensión uniformes.	`IPlugin` establece métodos clave (`initialize`, `start`, `process`, `stop` y `get_widget`) que todo plugin debe implementar.
Task orchestration / Executor pattern	Gestionar cálculos pesados sin bloquear la interfaz.	El Kernel centraliza la ejecución de tareas, su cola, cancelación, seguimiento del progreso y entrega de resultados.

5. Vistas Arquitectónicas (4+1)

5.1. Vista de Escenarios

5.1.1. Propósito

El diagrama de casos de uso describe el comportamiento observable de Gamma Lab 2.0 desde la perspectiva de su usuario. Representa las funcionalidades que el sistema ofrece, el actor que las utiliza y las relaciones de dependencia, extensión y especialización entre ellas. Cada caso de uso agrupa uno o más requerimientos funcionales, por lo que el modelo vincula la especificación de requerimientos con la estructura de la arquitectura. Además, al organizar los casos de uso en subsistemas, el diagrama anticipa la descomposición funcional del sistema y muestra cómo se distribuyen las responsabilidades entre sus componentes.

5.1.2. Notación

El diagrama sigue la notación de casos de uso definida en la especificación UML 2.5.1 del Object Management Group [17]. El sistema se representa como un contenedor general con el estereotipo «system». Dentro de él, los casos de uso se agrupan en paquetes con el estereotipo «subsistema», cada uno asociado a un área de responsabilidad funcional. Las relaciones entre casos de uso son de tres tipos:

- **Inclusión («include»):** línea punteada que va del caso base al caso incluido. Indica que el comportamiento incluido es obligatorio y forma parte de la ejecución del caso base.
- **Extensión («extend»):** línea punteada que va del caso extensor al caso base. Indica un comportamiento opcional o condicional que amplía el flujo del caso base sin modificarlo.
- **Generalización:** línea continua que va del caso especializado al caso general. Indica que el caso hijo hereda y especializa el comportamiento del padre.

Los casos de uso marcados con el estereotipo «abstract» no se ejecutan directamente. Su función es definir un comportamiento común que concretan sus especializaciones.

5.1.3. Visión general del modelo

El diagrama sigue el flujo de interacción de Gamma Lab 2.0, desde que el usuario inicia la aplicación hasta que obtiene sus resultados. El sistema tiene un único actor, el Investigador, que interactúa exclusivamente con la capa de presentación. Esta capa es el único punto de entrada al sistema y concentra la navegación entre módulos y la visualización de las señales. Después de cargar una señal o abrir un proyecto, el investigador puede acondicionarla con las operaciones de preprocesamiento y luego aplicarle los análisis disponibles. La interfaz no ejecuta directamente ninguna de estas operaciones, sino que las delega en el orquestador del núcleo. Este componente actúa como mediador entre la presentación y la lógica de procesamiento: recibe las solicitudes de procesamiento pesado, administra su ciclo de ejecución y permite cancelarlas. Gracias a esta mediación, la interfaz queda desacoplada de los algoritmos de procesamiento, cada subsistema conserva una responsabilidad bien definida y el control de la ejecución queda centralizado en el núcleo, como establece el estilo arquitectónico de microkernel. Al final del flujo, los resultados se persisten o se exportan, y el proyecto se cierra junto con la aplicación. Este recorrido se refleja en la organización del modelo en cinco subsistemas: Interfaz y Usabilidad, Núcleo y Ciclo de Vida, Preprocesamiento, Análisis y Exportación.

Las secciones siguientes describen cada uno. La Figura 2 presenta el diagrama de casos de uso del sistema.

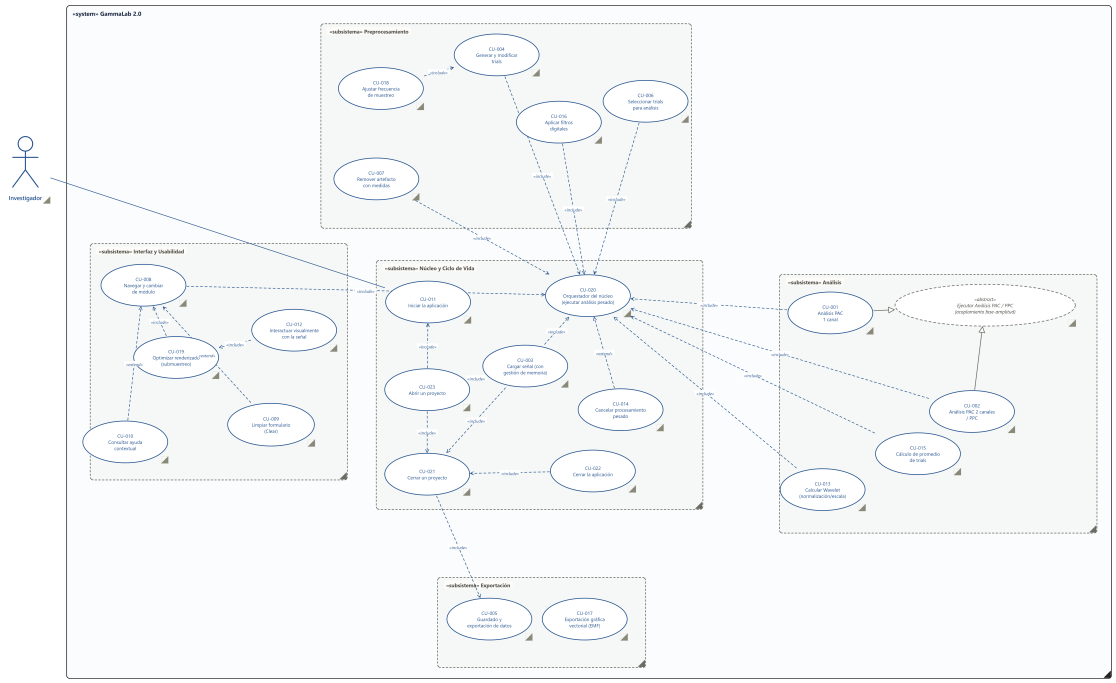


Figura 2: Diagrama de casos de uso del sistema completo Gamma Lab 2.0 (23 CU en 5 subsistemas). Las relaciones *include* / *extend* se tomaron literalmente del campo "Extensiones" de cada ficha; ninguna es inferida por similitud temática.

La Figura 2 revela que el subsistema **Núcleo y Ciclo de Vida** — en particular CU-020, el orquestador de tareas — es el verdadero centro de gravedad arquitectónico: diez de los otros veintidós casos de uso lo incluyen explícitamente. Esa centralidad se concreta a continuación en tres escenarios representativos.

5.1.4. Subsistema Interfaz y Usabilidad

El subsistema **Interfaz y Usabilidad** agrupa los casos de uso que definen cómo interactúa el Investigador con Gamma Lab 2.0 a través de la capa de presentación. Su responsabilidad no es procesar señales, sino garantizar que la interacción sea comprensible, fluida y estable en el uso de recursos. Por eso concentra principalmente requerimientos no funcionales de usabilidad, rendimiento y confiabilidad, junto con algunos requerimientos funcionales de visualización. El Cuadro 10 resume los casos de uso que lo componen.

Cuadro 10: Casos de uso del subsistema Interfaz y Usabilidad

Caso de uso	Nombre	Descripción
CU-008	Navegar y cambiar de módulo	Permite desplazarse entre los módulos funcionales del sistema, liberando los recursos del módulo saliente y conservando la señal activa en memoria.
CU-009	Limpiar formulario (Clear)	Restablece a sus valores por defecto los parámetros de una funcionalidad, sin modificar los resultados ya presentados.
CU-010	Consultar ayuda contextual	Muestra una descripción emergente del propósito de cada control de la interfaz.
CU-012	Interactuar visualmente con la señal	Permite explorar señales y resultados mediante zoom, lectura de coordenadas y vista Butterfly, sin alterar los datos.
CU-019	Optimizar renderizado (submuestreo)	Reduce la densidad de la representación gráfica para mantener la fluidez del renderizado, sin modificar los datos en memoria.

El subsistema se organiza alrededor de CU-008, que funciona como eje de la interacción y como único punto de conexión con el subsistema Núcleo y Ciclo de Vida. Esta posición se debe a que el cambio de módulo es una transición en el ciclo de vida de las extensiones. Al desactivarse, cada módulo debe liberar sus recursos gráficos y sus detectores de eventos de forma determinista. Al activarse, el módulo destino debe reutilizar los componentes ya construidos, sin recargar la señal desde disco. Los demás casos de uso del subsistema se vinculan con este eje. CU-010 lo extiende como un comportamiento opcional, activado por el usuario, que no altera el estado del sistema. CU-019 se relaciona con la navegación porque la optimización del renderizado condiciona la fluidez de cualquier vista presentada.

El diagrama también muestra un segundo agrupamiento, formado por CU-012 y CU-019. CU-012 incluye a CU-019, lo que expresa que toda interacción visual sobre una señal pasa obligatoriamente por la estrategia de submuestreo de representación. Esta relación materializa la separación entre el modelo de datos y la vista: el zoom, el desplazamiento y la lectura de coordenadas modifican solo la representación, mientras que los datos en memoria permanecen intactos.

Por último, CU-009 aparece en el diagrama vinculado con CU-019 mediante una relación de extensión. Este caso de uso representa una operación sobre el estado del formulario de cada funcionalidad: restablece los parámetros a sus valores por defecto sin modificar los resultados ya presentados ni los datos en memoria. Su ejecución está condicionada a que no exista un cómputo en curso lanzado desde ese formulario, lo que mantiene la coherencia entre los parámetros mostrados y la tarea en ejecución.

En conjunto, el subsistema resuelve dentro de la capa de presentación los atributos de calidad más sensibles para el usuario: la capacidad de respuesta, la fluidez del renderizado y el consumo acotado de memoria y recursos gráficos. Al concentrar su conexión con el resto del sistema en CU-008, mantiene la lógica de interacción desacoplada de la lógica de procesamiento y delega la ejecución de las operaciones pesadas en el orquestador del núcleo.

5.1.5. Subsistema Núcleo y Ciclo de Vida

El subsistema **Núcleo y Ciclo de Vida** materializa el estilo arquitectónico de microkernel. En él se registran los servicios compartidos, se descubren las extensiones y se centraliza la ejecución

de las operaciones pesadas, por lo que concentra requerimientos de rendimiento, confiabilidad y soporte. El Cuadro 11 resume los casos de uso que lo componen.

Cuadro 11: Casos de uso del subsistema Núcleo y Ciclo de Vida

Caso de uso	Nombre	Descripción
CU-011	Iniciar la aplicación	Arranca el sistema, registra en el núcleo los servicios compartidos y descubre las extensiones disponibles.
CU-022	Cerrar la aplicación	Finaliza la ejecución liberando hilos, extensiones, recursos gráficos y memoria.
CU-023	Abrir un proyecto	Restaura un proyecto guardado, con su señal de origen y el trabajo registrado sobre ella.
CU-003	Cargar señal (con gestión de memoria)	Carga un archivo de señal como nuevo proyecto activo, liberando previamente la memoria del proyecto anterior.
CU-021	Cerrar un proyecto	Cierra el proyecto activo, ofrece guardarlo y libera la memoria asociada a la señal y sus derivados.
CU-020	Orquestador del núcleo (ejecutar análisis pesado)	Recibe, encola y ejecuta fuera del hilo de la interfaz las operaciones de cómputo pesado y publica sus resultados en el estado compartido.
CU-014	Cancelar procesamiento pesado	Interrumpe una tarea en curso, libera su hilo trabajador y descarta el resultado parcial.

El diagrama organiza estos casos de uso en tres agrupamientos que corresponden a tres ciclos de vida distintos. El primero es el ciclo de vida de la aplicación, formado por CU-011 y CU-022. CU-011 deja disponibles los servicios del núcleo mediante inversión de control, de modo que ninguna extensión los instancia por su cuenta, y registra las extensiones sin que sea necesario modificar el núcleo ni las ya existentes. CU-022 es su contraparte: garantiza que al terminar la ejecución no queden hilos, procesos ni contextos gráficos activos.

El segundo agrupamiento corresponde al ciclo de vida del proyecto y lo forman CU-023, CU-003 y CU-021. En el diagrama, CU-021 recibe relaciones de inclusión desde CU-023, CU-003 y CU-022. Esto lo convierte en el punto único de cierre del proyecto: toda operación que reemplace o termine el proyecto activo reutiliza el mismo comportamiento para ofrecer el guardado y liberar la memoria. A su vez, CU-023 incluye a CU-011 y a CU-003, lo que expresa que abrir un proyecto puede implicar iniciar la aplicación y que la restauración de la señal se apoya en el mismo mecanismo de carga. CU-021 también se relaciona con el subsistema Exportación mediante la inclusión de CU-005, del que obtiene la persistencia del proyecto.

El tercer agrupamiento corresponde al ciclo de vida de las tareas de cómputo y lo forman CU-020 y CU-014. CU-020 es el elemento central del subsistema y del modelo completo, ya que recibe las relaciones de inclusión de los casos de uso de Preprocesamiento y Análisis, además de la de CU-003. Su función es desacoplar a las extensiones de la gestión de la concurrencia. Las extensiones no administran hilos propios, sino que solicitan la ejecución al núcleo. El orquestador encola las solicitudes y las ejecuta en hilos trabajadores, mantiene la interfaz responsiva durante el cómputo y publica los resultados en el estado compartido mediante escritura sincronizada con un único propietario. CU-014 extiende a CU-020 como un comportamiento opcional, activado por el usuario, que propaga la cancelación a la tarea en ejecución y garantiza la liberación de su hilo trabajador, aunque el cómputo no haya terminado.

En conjunto, el subsistema concentra las responsabilidades transversales del sistema: la administración de servicios y extensiones, la gestión de memoria entre proyectos y el control de la

ejecución concurrente. Al centralizar estas responsabilidades en el núcleo, los demás subsistemas pueden limitarse a su lógica funcional, y la incorporación de nuevas extensiones se reduce a registrarlas en el núcleo y conectarlas al orquestador.

5.1.6. Subsistema Preprocesamiento

El subsistema **Preprocesamiento** agrupa las operaciones que acondicionan la señal antes de su análisis. Su responsabilidad es transformar la señal y sus segmentaciones sin comprometer la integridad de los datos originales, por lo que concentra requerimientos funcionales de transformación y requerimientos de soporte orientados al encapsulamiento del modelo de datos. El Cuadro 12 resume los casos de uso que lo componen.

Cuadro 12: Casos de uso del subsistema Preprocesamiento

Caso de uso	Nombre	Descripción
CU-004	Generar y modificar trials	Segmenta la señal en ensayos a partir de un canal de estímulo y permite editar o descartar trials ya generados.
CU-018	Ajustar frecuencia de muestreo	Corrige la frecuencia de muestreo de la señal activa y recalcula su eje temporal.
CU-016	Aplicar filtros digitales	Aplica un filtro Butterworth pasa-banda de fase cero y registra el resultado como una señal derivada.
CU-007	Remover artefacto con medidas	Elimina artefactos de estimulación en los trials, mostrando antes las medidas del segmento afectado.
CU-006	Seleccionar trials para análisis	Define el subconjunto de trials que participa en un análisis y deja constancia de cuáles se usaron.

En el diagrama, todos los casos de uso del subsistema, con excepción de CU-018, mantienen una relación de inclusión con el orquestador del núcleo (CU-020). Esto expresa que ninguna transformación se ejecuta en el hilo de la interfaz: el subsistema define qué operación se aplica sobre la señal, y el núcleo determina cómo y cuándo se ejecuta. Así, las extensiones de preprocesamiento contienen solo su lógica de transformación y quedan desacopladas de la gestión de la concurrencia.

La única relación interna del subsistema es la inclusión de CU-004 por parte de CU-018. Como los trials se segmentan sobre el eje temporal de la señal, un cambio en la frecuencia de muestreo deja desactualizadas las segmentaciones existentes, y el sistema ofrece regenerarlas con el mismo mecanismo de corte. Por esta vía, CU-018 llega al orquestador de forma indirecta, a través de CU-004.

Más allá de las relaciones representadas, los casos de uso del subsistema comparten un mismo principio de diseño: los datos originales no se modifican. Cada transformación se registra como un conjunto derivado de la señal o de los trials, y el acceso al modelo se hace siempre a través de su API pública, sin que ninguna extensión manipule su estructura interna. Cuando un cambio afecta a datos ya representados o analizados, el sistema invalida las cachés dependientes y advierte qué resultados quedaron desactualizados. Con este principio se conserva la trazabilidad entre la señal original y cada una de sus transformaciones, y se reduce el acoplamiento entre las extensiones y el modelo de datos.

En conjunto, el subsistema actúa como la etapa de acondicionamiento previa al análisis. Combina

la delegación de la ejecución en el núcleo con la preservación de los datos originales, de modo que el investigador puede probar distintas transformaciones sin perder la señal de partida.

5.1.7. Subsistema Análisis

El subsistema **Análisis** agrupa los cómputos que extraen información de la señal acondicionada, como el acoplamiento entre bandas de frecuencia, la representación tiempo-frecuencia y el promedio de ensayos. Concentra requerimientos funcionales de cálculo y visualización de resultados, junto con requerimientos de rendimiento que acotan el tiempo de los cómputos más costosos. El Cuadro 13 resume los casos de uso que lo componen.

Cuadro 13: Casos de uso del subsistema Análisis

Caso de uso	Nombre	Descripción
CU-001	Análisis PAC de un canal	Calcula el acoplamiento fase-amplitud entre dos bandas de frecuencia de un mismo canal, sobre un trial único o el promedio de trials.
CU-002	Análisis PAC de dos canales / PPC	Calcula el acoplamiento fase-amplitud entre dos canales distintos, o la coherencia de fase entre ellos.
CU-013	Calcular Wavelet	Calcula la transformada wavelet de la señal con normalización y escala configurables, y presenta el escalograma tiempo-frecuencia.
CU-015	Cálculo de promedio de trials	Promedia todos los trials o un subconjunto seleccionado para reducir el ruido de la señal.

En el diagrama, todos los casos de uso concretos del subsistema mantienen una relación de inclusión con el orquestador del núcleo (CU-020). Esto es consecuente con la naturaleza del subsistema, que reúne los cómputos más costosos del sistema. Delegar su ejecución en el núcleo permite que la interfaz siga respondiendo durante el cálculo, que el usuario pueda cancelarlo y que los resultados se publiquen en el estado compartido de forma sincronizada. Cada extensión de análisis queda así limitada a su lógica de cálculo, sin intervenir en la gestión de la concurrencia.

La relación interna del subsistema es una generalización: CU-001 y CU-002 especializan el caso de uso abstracto Ejecutar análisis PAC / PPC, que define el procedimiento común de cálculo del acoplamiento fase-amplitud y sus visualizaciones. Las especializaciones se distinguen por el origen de los datos. CU-001 toma la fase y la amplitud de un mismo canal, y CU-002 las toma de canales distintos y añade la coherencia de fase. Esta estructura permite reutilizar el comportamiento común y agregar nuevas variantes de acoplamiento sin modificar las existentes.

CU-013 y CU-015 no participan en esta jerarquía y se representan como análisis independientes. CU-013 produce la representación tiempo-frecuencia de la señal y tiene un tiempo máximo de cómputo definido. CU-015 produce la serie promediada de los trials y opera sobre el subconjunto seleccionado en el subsistema Preprocesamiento.

En conjunto, el subsistema consume los datos acondicionados por el preprocesamiento y produce los resultados que después se visualizan y exportan. Al delegar la ejecución en el núcleo y organizar los análisis de acoplamiento mediante generalización, el subsistema puede crecer con nuevos análisis sin afectar la estabilidad de la interfaz ni la estructura de los análisis existentes.

5.1.8. Subsistema Exportación

El subsistema **Exportación** agrupa los casos de uso que trasladan el trabajo realizado en Gamma Lab 2.0 al sistema de archivos local. Su responsabilidad es la persistencia del proyecto y la exportación de resultados numéricos y gráficos, por lo que concentra requerimientos funcionales de formato y nomenclatura de archivos, junto con requerimientos de rendimiento que acotan el tiempo de cada exportación. El Cuadro 14 resume los casos de uso que lo componen.

Cuadro 14: Casos de uso del subsistema Exportación

Caso de uso	Nombre	Descripción
CU-005	Guardado y exportación de datos	Guarda el proyecto en formato .glab o exporta los resultados del análisis activo en formatos tabulares (Excel, CSV, JSON) o de imagen rasterizada (JPEG, PNG).
CU-017	Exportación gráfica vectorial (EMF)	Exporta las visualizaciones de los análisis en formato vectorial, conservando sus ejes y escalas para uso en publicaciones.

En el diagrama, el subsistema se conecta con el resto del sistema mediante una única relación: la inclusión de CU-005 por parte de CU-021, Cerrar un proyecto, del subsistema Núcleo y Ciclo de Vida. Esta relación expresa que el guardado del proyecto no es una operación aislada, sino una parte obligatoria del ciclo de vida del proyecto: toda operación que cierre o reemplace el proyecto activo pasa por el mismo mecanismo de persistencia. El archivo de proyecto almacena una referencia a la señal de origen junto con los trials, los análisis y sus parámetros, sin duplicar los datos de la señal. Así, el proyecto se mantiene liviano y puede restaurarse a partir del archivo original.

CU-017 se representa sin relaciones con otros casos de uso, porque es una operación que el usuario invoca directamente sobre una visualización ya generada. Complementa a CU-005 con un formato vectorial que conserva cada elemento gráfico por separado, en lugar de una imagen plana. Los dos casos de uso comparten las mismas convenciones de salida: el nombre sugerido del archivo incluye el nombre de la señal original y los tiempos de exportación están acotados según el tipo de contenido.

En conjunto, el subsistema cierra el flujo de trabajo del sistema, ya que convierte los resultados producidos por los subsistemas Preprocesamiento y Análisis en artefactos persistentes y reutilizables fuera de la aplicación. Al integrarse con el núcleo a través del cierre del proyecto, el sistema asegura que el trabajo del investigador pueda conservarse antes de liberar los recursos de la sesión.

5.2. Vista Lógica

5.2.1. Propósito

La vista lógica describe la descomposición funcional de Gamma Lab 2.0 en clases, interfaces y abstracciones, junto con las responsabilidades de cada elemento y las relaciones estructurales que los vinculan. Esta vista describe los elementos que lo componen, la responsabilidad que asume cada uno de ellos y la forma en que colaboran para satisfacer los requerimientos funcionales.

5.2.2. Notación

Dado que Gamma Lab 2.0 evoluciona a partir de un sistema existente, cada elemento del modelo se clasifica según su estado de implementación. Los elementos representados en color verde se encontraban implementados en el código fuente de gammaLab 1.0, mientras que los elementos representados en color rojo están planificados para gammaLab 2.0 y deben construirse para satisfacer casos de uso o requerimientos pendientes. Esta distinción permite que el modelo sirva simultáneamente como descripción del sistema pre-existente y como especificación de la arquitectura objetivo, es decir gammaLab 2.0.

5.2.3. Visión general del modelo

La estructura lógica de Gamma Lab 2.0 responde al estilo arquitectónico de microkernel, en el cual un núcleo mínimo administra un conjunto de extensiones independientes que aportan la funcionalidad del sistema. Los elementos del modelo se organizan en seis agrupaciones. El núcleo administra el ciclo de vida de las extensiones y el registro de servicios. Los servicios del sistema proveen capacidades transversales de configuración, entrada y salida de archivos, exportación, persistencia de proyectos, estado compartido y ejecución de tareas. La capa de presentación agrupa los elementos de interfaz gráfica que coordinan la interacción con el usuario. El contrato de extensión define la interfaz que todo módulo funcional debe cumplir. Las extensiones implementan las operaciones y los análisis sobre las señales. Finalmente, el modelo de datos representa las señales, sus segmentaciones y los proyectos persistidos.

La Figura 3 presenta el diagrama de clases del sistema.

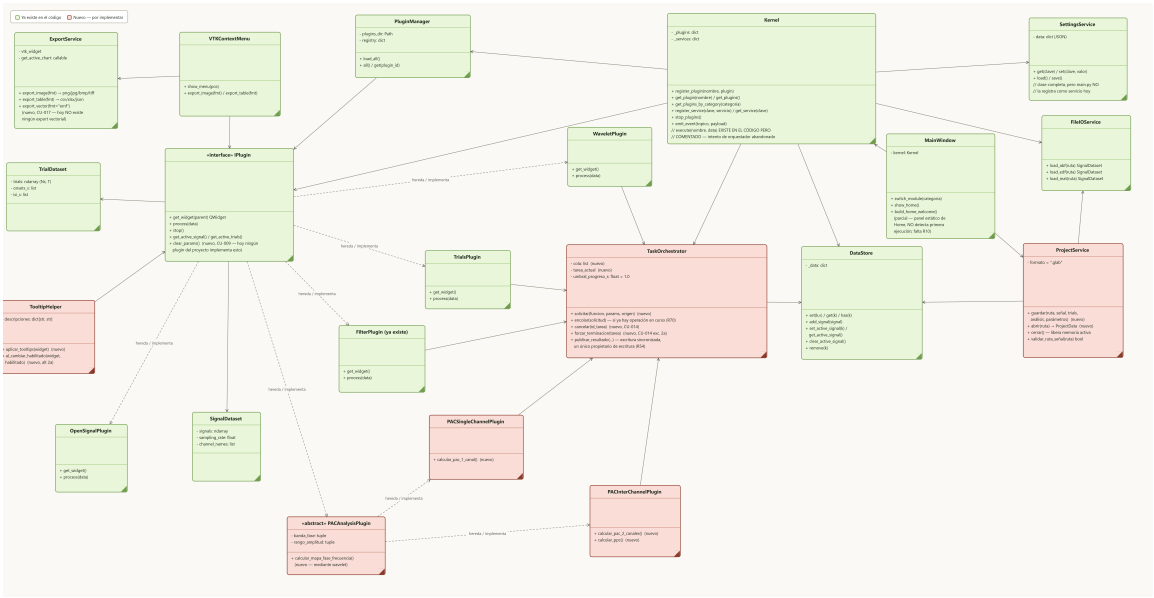


Figura 3: Diagrama de clases del sistema Gamma Lab 2.0. Los elementos en verde se encuentran implementados y los elementos en rojo están planificados.

5.2.4. Elementos del núcleo

Kernel El Kernel constituye el punto central de coordinación del sistema y se encuentra implementado. Mantiene dos registros internos: el registro de extensiones, indexadas por nombre, y el registro de servicios, indexados por clave. Sus operaciones permiten registrar una extensión (register_plugin), consultarla de forma individual o colectiva (get_plugin y get_plugins)

y obtener las extensiones pertenecientes a una categoría funcional (get_plugins_by_category), lo cual permite a la interfaz organizar los módulos por categorías. Asimismo, ofrece operaciones para registrar y localizar servicios por clave (register_service y get_service), para detener de forma ordenada todas las extensiones al finalizar la aplicación (stop_plugins) y para publicar eventos dirigidos a los componentes suscritos (emit_event). El Kernel se asocia con el PluginManager, el SettingsService, el FileIOService, el DataStore y el TaskOrchestrator, y administra las extensiones exclusivamente a través de la interfaz IPlugin.

PluginManager El PluginManager se encuentra implementado y es responsable de descubrir, cargar e instanciar las extensiones disponibles en el directorio de extensiones del sistema. Conserva la ubicación de dicho directorio y un registro de las extensiones descubiertas. Sus operaciones permiten cargar la totalidad de las extensiones (load_all) y consultarlas de forma colectiva o por identificador (all y get). El PluginManager depende de la interfaz IPlugin, de modo que únicamente admite elementos que cumplan dicho contrato.

5.2.5. Servicios del sistema

SettingsService El SettingsService se encuentra implementado y gestiona la configuración persistente de la aplicación. La configuración se mantiene en memoria como un diccionario y se almacena en disco en formato JSON. El servicio ofrece operaciones para consultar y modificar valores de configuración por clave (get y set) y para cargar y almacenar la configuración (load y save).

FileIOService El FileIOService se encuentra implementado y adapta los formatos de archivo soportados al modelo de datos interno del sistema. Sus operaciones load_abf, load_edf y load_mat leen archivos en los formatos ABF, EDF y MAT, respectivamente, y retornan en todos los casos un objeto SignalDataset. De esta manera, el resto del sistema permanece independiente de la representación de las señales en disco.

ExportService El ExportService se encuentra implementado y es responsable de exportar visualizaciones y resultados tabulares a formatos externos. Mantiene una referencia al componente de visualización y una función que provee la gráfica activa en cada momento. La operación export_image exporta la visualización a formatos de imagen ráster (PNG, JPG, BMP y TIFF) y la operación export_table exporta los resultados a formatos tabulares (CSV, XLSX y JSON). Se encuentra planificada la operación export_vector, que permitirá la exportación a formato vectorial EMF en respuesta al caso de uso CU-017.

DataStore El DataStore se encuentra implementado y mantiene en memoria el estado compartido de la sesión de trabajo, el cual comprende las señales cargadas, la señal activa, las mediciones y los resultados de los análisis. Constituye la fuente única de verdad del estado de la aplicación. Ofrece operaciones genéricas de almacenamiento, consulta y verificación por clave (set, get y has), operaciones para incorporar señales (add_signal), para establecer, consultar y liberar la señal activa (set_active_signal, get_active_signal y clear_active_signal) y para eliminar entradas del estado (remove).

TaskOrchestrator El TaskOrchestrator tiene la responsabilidad de coordinar la ejecución de las operaciones de cómputo solicitadas por las extensiones. Su propósito es garantizar que dichas operaciones se ejecuten de forma ordenada, que puedan ser canceladas por el usuario, que se

presente retroalimentación de progreso y que sus resultados se publiquen de manera consistente. Para ello mantendrá una cola de solicitudes pendientes, una referencia a la operación en ejecución y un umbral de tiempo, fijado en un segundo, a partir del cual se presentará el indicador de progreso.

La operación solicitar constituirá el punto de entrada único mediante el cual una extensión pide la ejecución de un cómputo. La operación encolar diferirá una solicitud cuando exista otra operación en curso, en cumplimiento del requerimiento R70. Las operaciones cancelar y forzar_terminacion permitirán, respectivamente, la cancelación de una tarea a petición del usuario y la terminación forzada de una tarea que no responda a la cancelación, conforme al caso de uso CU-014 y su excepción 2a. Por último, la operación publicar_resultado escribirá los resultados en el estado compartido de forma sincronizada. El TaskOrchestrator será el único componente autorizado para escribir en el DataStore, en cumplimiento del requerimiento R54.

ProjectService El ProjectService se encuentra implementado y gestiona la persistencia del espacio de trabajo en archivos de proyecto con extensión glab. Cada archivo de proyecto es un manifiesto en formato JSON que registra la señal de origen, las mediciones realizadas, la configuración de la segmentación en ensayos y el estado de los análisis. Durante el guardado, el archivo de señal se copia a la carpeta del proyecto, de modo que el proyecto resulta autocontenido y puede reabrirse en otro equipo. Durante la apertura, el servicio verifica la disponibilidad del archivo de señal referenciado, lo recarga mediante el FileIOService y restaura el estado de la sesión en el DataStore.

El servicio mantiene la carpeta del proyecto, la ruta del archivo de proyecto, el nombre del proyecto, un indicador de cambios pendientes de guardar y una función de notificación que informa a la interfaz sobre la apertura, el guardado, el cambio de nombre o el cierre del proyecto. Sus operaciones principales permiten guardar el proyecto en su ubicación actual (save), guardarlo en una nueva ubicación (save_as), abrir y restaurar un proyecto existente (open), y cerrar el proyecto activo liberando su estado (close). Adicionalmente, ofrece operaciones para asignar el nombre del proyecto (set_project_name), registrar la existencia de cambios pendientes (mark_dirty), verificar si existe un proyecto abierto (is_open) y generar un nombre predeterminado a partir del nombre de la señal (default_project_name). El ProjectService depende del FileIOService, del DataStore y del Kernel, y produce y consume objetos de tipo ProjectData.

5.2.6. Elementos de presentación

MainWindow La MainWindow se encuentra parcialmente implementada y constituye el contenedor principal de la interfaz gráfica. Es responsable de la navegación entre los módulos funcionales, de la presentación de la pantalla de inicio y de las operaciones de proyecto disponibles en el menú Archivo. Mantiene referencias al Kernel y al ProjectService. La operación switch_module presenta las extensiones correspondientes a la categoría seleccionada por el usuario, la operación (show_home) presenta la pantalla de inicio y la operación (build_home_welcome) construye el panel de bienvenida. Esta última se encuentra pendiente de incorporar la detección de la primera ejecución de la aplicación, requerida por el requerimiento R10.

VTKContextMenu El VTKContextMenu se encuentra implementado y provee el menú contextual asociado a las visualizaciones. Sus operaciones permiten presentar el menú en la posición indicada por el usuario (show_menu) y solicitar la exportación de la visualización como imagen o como tabla (export_image y export_table). Este elemento delega la exportación en el ExportService y opera sobre los componentes visuales provistos por las extensiones.

TooltipHelper El TooltipHelper se encuentra planificado y tendrá la responsabilidad de proveer ayuda contextual sobre los controles de la interfaz. Mantendrá un diccionario de descripciones asociadas a cada control. La operación (aplicar_tooltips) asignará dichas descripciones a los controles de un componente visual y la operación (al_cambiar_habilitado) actualizará la descripción de un control cuando cambie su disponibilidad, de modo que el usuario conozca el motivo por el cual un control se encuentra deshabilitado.

5.2.7. Contrato de extensión

IPlugin IPlugin es la interfaz que define el contrato que toda extensión debe cumplir para integrarse al sistema, y constituye el mecanismo principal de extensibilidad de la arquitectura. Se encuentra implementada y establece las siguientes operaciones: get_widget, que provee la interfaz gráfica de la extensión; process, que ejecuta el procesamiento propio de la extensión; stop, que detiene la extensión y libera sus recursos; y get_active_signal y get_active_trials, que proporcionan acceso a los datos de trabajo. IPlugin depende de las clases SignalDataset y TrialDataset.

Se encuentra planificada la incorporación al contrato de la operación clear_params, cuya finalidad es restablecer los parámetros de una extensión a sus valores predeterminados, conforme al caso de uso CU-009. Actualmente esta operación se encuentra implementada de forma individual en las extensiones FilterPlugin, TrialsPlugin y ArtifactRemovePlugin, pero aún no forma parte de la interfaz común.

5.2.8. Extensiones

Todas las extensiones del sistema realizan la interfaz IPlugin. La Tabla 15 resume las extensiones representadas en el modelo, su estado de implementación y su responsabilidad.

Cuadro 15: Extensiones representadas en la vista lógica.

Elemento	Responsabilidad
`OpenSignalPlugin`	Selección y carga de señales desde archivo.
`FilterPlugin`	Filtrado en frecuencia de la señal.
`WaveletPlugin`	Análisis en tiempo y frecuencia mediante la transformada wavelet.
`TrialsPlugin`	Segmentación de la señal en ensayos.
`PACAnalysisPlugin`	Comportamiento común del análisis de acoplamiento entre fase y amplitud (PAC).
`PACSingleChannelPlugin`	Cálculo del acoplamiento entre fase y amplitud dentro de un mismo canal.
`PACInterChannelPlugin`	Cálculo del acoplamiento entre fase y amplitud y de la consistencia de fase por pares (PPC) entre dos canales.

PACAnalysisPlugin es una clase abstracta que concentra los parámetros y el procedimiento comunes a todo análisis de acoplamiento entre fase y amplitud. Sus atributos definen la banda de frecuencia de la fase y el rango de frecuencia de la amplitud, y su operación calcular_mapa_fase_frecuencia obtiene el mapa de fase y frecuencia mediante la transformada wavelet. Las clases PACSingleChannelPlugin y PACInterChannelPlugin especializan este comporta-

miento. La primera incorpora la operación calcular_pac_1_canal, que calcula el acoplamiento dentro de un mismo canal, y la segunda incorpora las operaciones calcular_pac_2_canales y calcular_ppc, que calculan el acoplamiento y la consistencia de fase entre dos canales.

5.2.9. Modelo de datos

SignalDataset SignalDataset se encuentra implementada y representa una señal electrofisiológica multicanal. Sus atributos contienen la matriz de muestras de la señal, la frecuencia de muestreo expresada en hercios y los identificadores de los canales.

TrialDataset TrialDataset se encuentra implementada y representa una señal segmentada en ensayos. Sus atributos contienen la matriz de ensayos, cuyas dimensiones corresponden al número de muestras por ensayo y al número de ensayos, los instantes de inicio de cada ensayo expresados en segundos y los intervalos entre estímulos expresados en segundos.

ProjectData ProjectData es una estructura de datos implementada que representa en memoria el contenido de un archivo de proyecto. En la implementación actual se materializa como un diccionario serializado en formato JSON, producido por la operación save y retornado por la operación open del ProjectService. Sus atributos registran la versión del esquema del formato de proyecto, el nombre del proyecto, la fecha y hora del último guardado, la información de la señal de origen (nombre de archivo, formato, nombre y frecuencia de muestreo), las mediciones realizadas, la configuración de la segmentación en ensayos y los parámetros y el estado de las extensiones de análisis. La versión del esquema permite detectar incompatibilidades entre versiones del formato al momento de abrir un proyecto.

5.2.10. Relaciones estructurales

La Tabla 16 resume las relaciones entre los elementos del modelo lógico.

Cuadro 16: Relaciones estructurales de la vista lógica.

Origen	Tipo	Destino	Descripción
Kernel	Asociación	PluginManager	Delega la carga de extensiones.
Kernel	Asociación	SettingsService, FileIOService, DataStore, TaskOrchestrator	Registra y expone los servicios del sistema.
Kernel, PluginManager	Dependencia	IPlugin	Administran las extensiones a través de su contrato.
MainWindow	Asociación	Kernel, ProjectService	Accede a las extensiones, a los servicios y a las operaciones de proyecto.
ProjectService	Dependencia	FileIOService, DataStore	Recarga la señal y restaura el estado de la sesión.
ProjectService	Dependencia	ProjectData	Produce y consume el contenido del proyecto.
VTKContextMenu	Dependencia	ExportService	Delega la exportación.
VTKContextMenu, TooltipHelper	Dependencia	IPlugin	Operan sobre los componentes visuales de las extensiones.
IPlugin	Dependencia	SignalDataset, TrialDataset	Manipula el modelo de datos.
Extensiones concretas y PACAnalysisPlugin	Realización	IPlugin	Implementan el contrato de extensión.
PACSingleChannelPlugin, PACInterChannelPlugin	Generalización	PACAnalysisPlugin	Especializan el análisis de acoplamiento.
Extensiones de análisis	Dependencia	TaskOrchestrator	Solicitan la ejecución de operaciones de cómputo.
TaskOrchestrator	Asociación	DataStore	Publica los resultados en el estado compartido.

5.2.11. Justificación de las decisiones

La separación entre el núcleo y las extensiones mediante la interfaz IPlugin favorece la extensibilidad del sistema, ya que permite incorporar nuevos análisis, como los de acoplamiento entre fase y amplitud, sin modificar el código existente, en conformidad con el principio de abierto y cerrado.

La asignación exclusiva de la escritura sobre el estado compartido al TaskOrchestrator garantiza la consistencia del estado de la aplicación, al eliminar las condiciones de carrera entre operaciones concurrentes, conforme al requerimiento R54.

La centralización de la ejecución de los cómputos en el TaskOrchestrator mejora la capacidad de respuesta del sistema y el control que ejerce el usuario sobre él, puesto que habilita la ejecución ordenada de las operaciones (R70), su cancelación (CU-014) y la presentación de progreso, sin que cada extensión deba gestionar sus propios hilos de ejecución.

La concentración de la lectura de archivos en el FileIOService asegura la independencia del sistema respecto de los formatos de señal, y confina a un único elemento el impacto de incorporar nuevos formatos.

La persistencia de proyectos mediante el ProjectService y la estructura ProjectData produce proyectos autocontenidos y versionados. La inclusión de una copia de la señal de origen permite reabrir el trabajo en otro equipo, y la versión del esquema permite detectar incompatibilidades entre versiones del formato.

Por último, la jerarquía encabezada por PACAnalysisPlugin promueve la reutilización, al evitar la duplicación del cálculo del mapa de fase y frecuencia entre las variantes de análisis de uno y de dos canales.

5.2.12. Trazabilidad con los requerimientos

La Tabla 17 relaciona los elementos y operaciones del modelo lógico con los casos de uso y requerimientos que los originan.

Cuadro 17: Trazabilidad entre elementos de la vista lógica y requerimientos.

Elemento u operación	**Caso de uso o requerimiento**
IPlugin, operación clear_params	CU-009
TaskOrchestrator, operación cancelar	CU-014
TaskOrchestrator, operación forzar_terminacion	CU-014, excepción 2a
ExportService, operación export_vector	CU-017
ProjectService, operaciones save, save_as y close	CU-021, R07
ProjectService, operación open, y ProjectData	CU-023
TaskOrchestrator, operación publicar_resultado	R54
TaskOrchestrator, operación encolar	R70
MainWindow, operación build_home_welcome	R10
PACSingleChannelPlugin y PACInterChannelPlugin	CU-001 y CU-002

5.3. Vista de Procesos

La vista de procesos describe el comportamiento dinámico del sistema completo como un único flujo maestro de sesión, construido a partir de las cadenas de precondición declaradas por las propias 23 fichas (p. ej. CU-006 exige trials generados por CU-004, que a su vez exige una señal cargada por CU-003) — no es una fusión literal de los 23 flujos detallados, que sería ilegíble.

El flujo de la Figura 4 se organiza en las siguientes etapas:

1. **Inicio.** El sistema registra los servicios del núcleo y descubre los plugins vía YAML (CU-011); el registro del servicio de tareas en este paso es nuevo, el resto ya existe en `main.py`.
2. **Apertura o carga.** Según si el investigador abre un `.glab` (CU-023, nuevo) o carga una señal desde Home vacío (CU-003, ya existe vía `FileIOService`), la señal activa termina publicándose en `DataStore`.
3. **Preprocesamiento.** Generación/edición de trials, selección de trials, remoción de artefactos, filtrado y ajuste de frecuencia de muestreo (CU-004, CU-006, CU-007, CU-016, CU-018) — los cuatro plugins ya existen en el código.
4. **Análisis.** PAC/PPC (CU-001, CU-002, nuevos), Wavelet (CU-013), promedio de trials (CU-015).
5. **Orquestación.** Todo cómputo pesado se despacha al orquestador del núcleo (CU-020, nuevo), cancelable en cualquier momento (CU-014, nuevo).
6. **Visualización.** Interacción visual con el resultado, con submuestreo si la señal es pesada

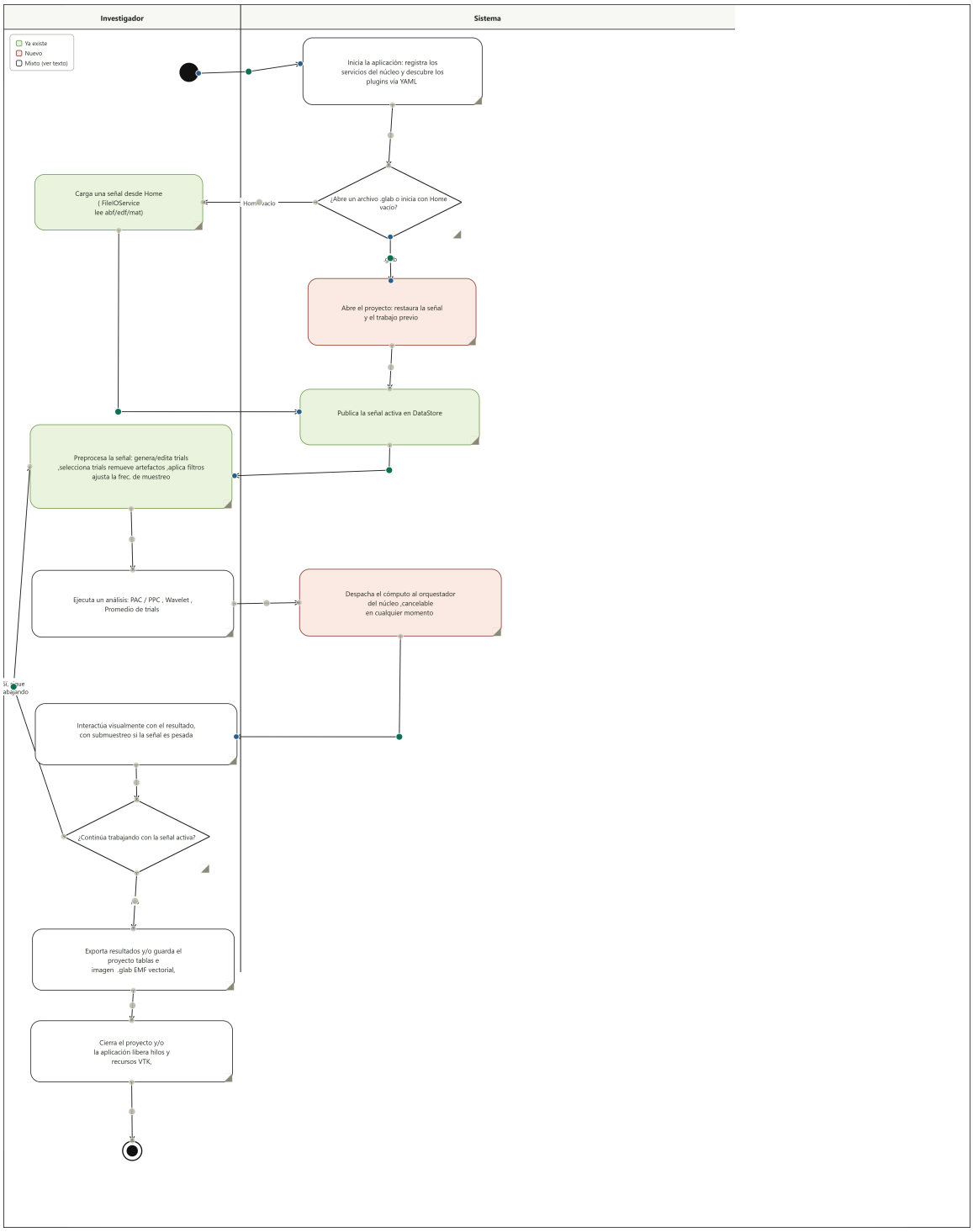


Figura 4: Flujo maestro de sesión de Gamma Lab 2.0, con carriles Investigador/Sistema. Verde = ya existe, rojo = nuevo, blanco = mixto.

(CU-012 parcial + CU-019, nuevo).

7. **Iteración o cierre.** Si el investigador continúa trabajando, el flujo retorna al preprocesamiento; de lo contrario, exporta y/o guarda el proyecto (CU-005 parcial, CU-017 nuevo) y cierra el proyecto y/o la aplicación (CU-021 nuevo, CU-022 parcial).

A diferencia de Gamma Lab 1.0, los casos de uso de navegación (CU-008), limpieza de formulario (CU-009, 0 % implementado) y ayuda contextual (CU-010, cobertura parcial en solo 2 de más de 15 plugins) son transversales a toda la sesión: no ocupan un lugar fijo en la secuencia y se documentan como notas adjuntas al diagrama en lugar de forzarlas dentro del flujo principal.

5.4. Vista Física

La vista física describe el entorno de despliegue local de Gamma Lab 2.0. A diferencia de Gamma Lab 1.0, aquí se documenta explícitamente la superficie completa de entrada/salida del sistema, incluyendo los artefactos que aún no existen.

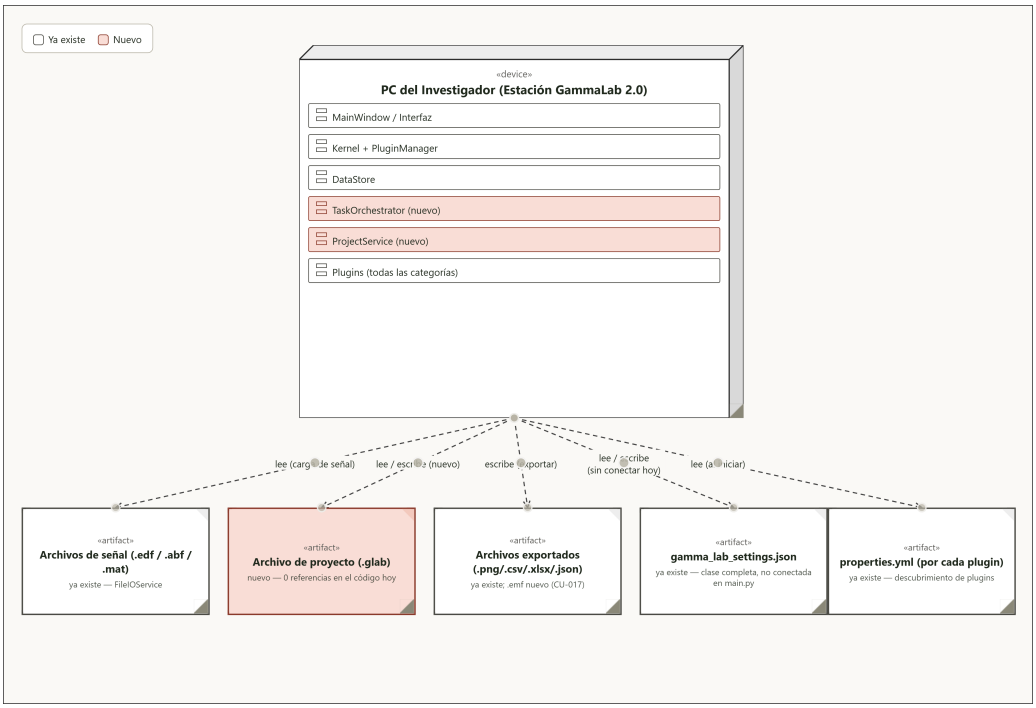


Figura 5: Vista física de Gamma Lab 2.0: estación única de escritorio y su superficie de E/S.

La Figura 5 muestra el despliegue típico:

- Un único nodo *PC del Investigador*: no existe arquitectura cliente-servidor en ninguna de las 23 fichas.
- **Archivos de señal** (`.edf`/`.abf`/`.mat`): ya existe, leídos por `FileIOService`.
- **Archivo de proyecto** (`.glab`): **nuevo** — cero referencias en el código actual.
- **Archivos exportados** (`.png`/`.csv`/`.xlsx`/`.json`): ya existe vía `ExportService`; el formato vectorial `.emf` (CU-017) es nuevo.
- `gamma_lab_settings.json`: la clase `SettingsService` ya existe, pero **no** está conectada en el arranque (`main.py`) hoy.
- `properties.yml` (por cada plugin): ya existe, sostiene el descubrimiento automático de plugins.

6. Atributos de calidad

6.1. Escenarios de calidad

6.1.1. Escenario de Calidad 1: Integración de un Plugin Externo

Cuadro 18: Escenario de calidad 1 — Integración de un plugin externo

Aspecto	**Descripción**
Atributo de calidad	Soporte
Requisito	El sistema debe permitir integrar plugins externos sin recompilar el núcleo y sin afectar módulos existentes.
Fuente del estímulo	Desarrollador externo
Estímulo	Se agrega un nuevo plugin a la carpeta de Plugins siguiendo el formato definido por el Kernel.
Artefacto	Módulo core del sistema (Kernel) y módulo de plugins.
Ambiente	Sistema en operación normal con múltiples plugins ya registrados.
Respuesta	El sistema detecta, valida y registra el plugin, exponiéndolo en la interfaz de usuario.
Medida de Respuesta	Plugin visible en la UI en menos de 1 minuto. Ningún plugin existente debe fallar tras la carga.
Trazabilidad	NFR-S-05 (R77–R78)

6.1.2. Escenario de Calidad 2: Carga de Señal de Gran Tamaño

Cuadro 19: Escenario de calidad 2 — Carga de señal de gran tamaño

Aspecto	**Descripción**
Atributo de calidad	Rendimiento
Requisito	El sistema debe gestionar señales de gran tamaño sin bloqueos ni degradación significativa del rendimiento.
Fuente del estímulo	Usuario final
Estímulo	Se carga un archivo ABF/EDF de gran tamaño (ej. 1–2 GB, múltiples canales).
Artefacto	FileIOService, DatasetService (DataStore), servicio de tareas del núcleo, interfaz de usuario.
Ambiente	Computador con carga moderada y otros procesos del sistema activos.
Respuesta	La señal se carga correctamente, sin duplicación innecesaria de datos y sin bloqueos de la interfaz; la lectura corre en el servicio de tareas, fuera del hilo de la UI.
Medida de Respuesta	Visualización inicial en menos de 8 segundos para este caso de estrés (el umbral base de carga es ≤ 2 s para el archivo típico de referencia). Uso de CPU menor a 80 %. Sin cierres inesperados ni latencias mayores a 5 segundos.
Trazabilidad	NFR-P-03 (R48), NFR-P-14 (R72), NFR-P-10 (R56)

6.1.3. Escenario de Calidad 3: Manejo de Errores en Tiempo de Ejecución

Cuadro 20: Escenario de calidad 3 — Manejo de errores en tiempo de ejecución

Aspecto	**Descripción**
Atributo de calidad	Confiabilidad
Requisito	El sistema debe manejar excepciones en los plugins sin comprometer la estabilidad del sistema.
Fuente del estímulo	Sistema o entorno
Estímulo	Un plugin genera una excepción interna durante un análisis (FFT, PSD, Wavelet, ERP, PAC, etc.).
Artefacto	Módulo de plugins, Kernel, servicios asociados a cada plugin.
Ambiente	Análisis en curso con señales cargadas y otros plugins activos.
Respuesta	El sistema captura la excepción, informa al usuario y evita un cierre inesperado de la aplicación, manteniendo el Kernel operando de manera adecuada.
Medida de Respuesta	0 % de pérdida de datos. Continuidad del sistema garantizada. Mensaje de error mostrado en menos de 2 segundos.
Trazabilidad	R47

6.1.4. Escenario de Calidad 4: Exportación de Datos o Gráficas

Cuadro 21: Escenario de calidad 4 — Exportación de datos o gráficas

Aspecto	**Descripción**
Atributo de calidad	Confiabilidad
Requisito	El sistema debe exportar gráficos y datos sin errores, incluso si se presentan fallas externas por disco, permisos, memoria u otros motivos.
Fuente del estímulo	Usuario final
Estímulo	El usuario exporta una tabla, medición o gráfico en PNG/CSV/JSON.
Artefacto	ExportService (registrado en el núcleo por inversión de control), interfaz de usuario, sistema de archivos.
Ambiente	Sistema operando con múltiples análisis realizados previamente sin guardar.
Respuesta	El archivo se genera correctamente o, en caso de error, se notifica al usuario sin afectar el estado del sistema.
Medida de Respuesta	Tiempo de exportación menor a 3 segundos. 0 % de pérdida de datos. Notificación clara ante error.
Trazabilidad	NFR-P-07 (R51), NFR-P-12 (R57), NFR-S-03 (R70)

6.1.5. Escenario de Calidad 5: Uso del Sistema sin Experiencia Técnica

Cuadro 22: Escenario de calidad 5 — Operación por un usuario sin experiencia técnica

Aspecto	**Descripción**
Atributo de calidad	Usabilidad
Requisito	El sistema debe ser utilizable por un usuario con experiencia en neurociencias o análisis de señales, pero sin conocimientos profundos del software.
Fuente del estímulo	Usuario final
Estímulo	El usuario utiliza un plugin complejo como Wavelet, PSD o PAC por primera vez.
Artefacto	Interfaz de usuario, ayudas contextuales, parámetros preconfigurados.
Ambiente	Uso sin acompañamiento técnico.
Respuesta	El usuario puede completar el análisis apoyándose en la interfaz, con guía clara y sin errores operativos.
Medida de Respuesta	Análisis realizado en menos de 5 minutos. Bajo número de consultas al manual de usuario.
Trazabilidad	NFR-U-07 (R42), NFR-U-08 (R43), NFR-U-05 (R10), NFR-U-09 (R44)

6.1.6. Escenario de Calidad 6: Rendimiento con Múltiples Señales Cargadas

Cuadro 23: Escenario de calidad 6 — Rendimiento con múltiples señales cargadas

Aspecto	**Descripción**
Atributo de calidad	Rendimiento
Requisito	El sistema debe mantener tiempos de respuesta estables aun cuando varias señales se encuentren cargadas simultáneamente dentro de la misma ProjectSession.
Fuente del estímulo	Usuario final
Estímulo	El usuario carga entre 2 y 4 señales de gran tamaño y ejecuta diferentes análisis en cada una.
Artefacto	DataStore, gestor de vistas, motor de análisis.
Ambiente	Sistema operando con alto volumen de datos en memoria.
Respuesta	El sistema reutiliza datos sin duplicarlos, mantiene tiempos de acceso aceptables y evita bloqueos en la interfaz.
Medida de Respuesta	Cambio entre señales cargadas en menos de 1 segundo. Sin bloqueos ni cierres inesperados. Uso de memoria dentro del 70–75 %.
Trazabilidad	NFR-P-06 (R51), NFR-P-01/P-08 (R11, R53)

6.1.7. Escenario de Calidad 7: Resiliencia ante Archivos Corruptos o Incompatibles

Cuadro 24: Escenario de calidad 7 — Resiliencia ante archivos corruptos o incompatibles

Aspecto	Descripción
Atributo de calidad	Confiabilidad
Requisito	El sistema debe detectar archivos corruptos, incompletos o incompatibles sin comprometer la estabilidad del programa.
Fuente del estímulo	Usuario final o sistema externo
Estímulo	El usuario intenta cargar un archivo ABF/EDF/MAT con estructura inválida o con metadatos corruptos.
Artefacto	FileIOService, Kernel, sistema de alertas.
Ambiente	Sistema en operación normal con señales previamente cargadas.
Respuesta	El sistema captura el error, notifica al usuario, cancela la operación de carga y mantiene intacta la sesión actual.
Medida de Respuesta	Mensaje de error mostrado en menos de 2 segundos. 0 % de pérdida de datos o señales cargadas previamente. El sistema continúa operativo sin fallos.
Trazabilidad	FR-IO-04

6.1.8. Escenario de Calidad 8: Consistencia entre Datos Ingresados y Gráficas Generadas

Cuadro 25: Escenario de calidad 8 — Consistencia entre datos ingresados y gráficas generadas

Aspecto	Descripción
Atributo de calidad	Confiabilidad y Usabilidad
Requisito	El sistema debe garantizar que las gráficas generadas correspondan fielmente a los datos y parámetros seleccionados por el usuario.
Fuente del estímulo	Usuario final
Estímulo	El usuario modifica parámetros como canal, ventana temporal, trials seleccionados o rango de frecuencias.
Artefacto	Interfaz de usuario, motor de visualización, servicios de análisis.
Ambiente	Sistema ejecutando análisis consecutivos sobre diferentes configuraciones.
Respuesta	La gráfica generada refleja exactamente los datos filtrados o seleccionados, sin desajustes ni inconsistencias con los parámetros ingresados.
Medida de Respuesta	Actualización gráfica en menos de 3 segundos. 0 diferencias entre los datos mostrados y los procesados. Consistencia confirmada entre ejecuciones sucesivas.
Trazabilidad	NFR-R-04 (R73), NFR-U-02 (R03)

6.2. Reglas de compatibilidad y versionado

- **Versionado de la aplicación.** GammaLab 2.0 adopta versionado semántico `MAJOR.MINOR.PATCH`: `MAJOR` rompe `.glab` o el contrato `IPlugin`; `MINOR` agrega capacidades retrocompatibles; `PATCH` corrige sin alterar contratos públicos. `main.py` expone esta versión en una constante única, visible en un diálogo “Acerca de” y embebida como metadato en cada archivo `.glab` generado.

- **Contrato de plugins (`IPlugin`).** Cambios aditivos (nuevos métodos opcionales, nuevos campos con valor por defecto) no rompen compatibilidad. Cambios que alteren la firma de `initialize`, `start`, `stop`, `process` o `get_widget` elevan la versión `MAJOR` del contrato.
- **Metadatos `properties.yml`.** Los campos requeridos (`id, name, category, subcategory, version, icon, logic_class, ui_class`) deben existir siempre – exigido por `REQUIRED_KEYS` en `loader.py` – y añadir campos opcionales es compatible. El campo `version` declara el `MAJOR` de `IPlugin` contra el que se escribió el plugin; `discover()` rechaza el plugin si no coincide con el `MAJOR` del núcleo en ejecución.
- **Formato de proyecto `.glab`.** `ProjectService` persiste el espacio de trabajo – señal activa, trials y análisis – en archivos `.glab`. Cada `.glab` embebe un `schema_version` propio: un incremento `MINOR` se mantiene legible por versiones futuras de `ProjectService`, y un incremento `MAJOR` rechaza la apertura con un mensaje explícito al usuario en vez de migrar implícitamente.
- **Dependencias externas.** `requirements.txt` fija cada librería con versión exacta (`==`), sin rangos abiertos (Tabla 26). Actualizar VTK, SciPy, PyWavelets o NumPy exige re-ejecutar completas las suites `plugins_test` y `performance_test` antes de aceptar el cambio.
- **Formatos de señal externos (EDF/ABF/BDF/MAT).** La compatibilidad se ancla a la versión de la librería lectora (`pyabf`, `pyedflib`), no al formato en sí. Un archivo no interpretable por esa versión se reporta como incompatible, sin remuestreo ni heurísticas de recuperación implícitas: un EDF con canales a distinta frecuencia de muestreo falla con un mensaje claro, en vez de intentar corregirse automáticamente.
- **Estabilidad de módulos internos.** Todo renombramiento o reubicación de un módulo público de `core/` actualiza en el mismo cambio a todos sus consumidores, tests incluidos, y un flujo de CI corre `pytest` en cada *push/pull request*. Motivo: un renombramiento previo de `fileio.py` a `fileio_service.py` (con `TrialDataset` movido a `core/model/`) rompió silenciosamente los 8 archivos de `plugins_test` al no actualizarse en el mismo cambio.

Cuadro 26: Versiones de dependencias externas ancladas en `requirements.txt`

Librería	Versión	Rol
`numpy`	2.3.4	Cálculo numérico vectorizado
`scipy`	1.16.2	Filtros, PSD (Welch)
`PyWavelets`	1.9.0	Transformada Wavelet Continua
`vtk`	9.5.2	Renderizado científico 2D/3D
`PyQt5` / `PyQt5-Qt5`	5.15.11 / 5.15.2	Interfaz gráfica
`pyabf`	2.3.8	Lectura de archivos `.abf`
`pyEDFlib`	0.1.42	Lectura de archivos `.edf`
`h5py`	3.15.1	Soporte de formatos basados en HDF5
`pytest`	8.4.2	Framework de pruebas

6.3. Métricas y plan de verificación

La Tabla 27 formaliza la "Medida de Respuesta" de cada escenario de calidad (sección 5.1) en una métrica verificable, junto con su método de medición y el criterio de aceptación correspondiente.

Cuadro 27: Métricas, método de verificación y criterio de aceptación por atributo de calidad

Atributo	Métrica	Método	Criterio de aceptación
Extensibilidad (Esc. 1)	Tiempo de aparición de un plugin nuevo; plugins existentes que fallan	Agregar un plugin válido y correr `plugins_test`	<1 min; 0 plugins fallan (R77–R78)
Rendimiento – carga grande (Esc. 2)	Tiempo a primer render; uso de CPU	`performance_test`; medido: 78.4 ms (1.8 M muestras)	≤8 s estrés, ≤2 s típico; CPU <80 % (R48/R72/R56)
Confiabilidad – errores en ejecución (Esc. 3)	% pérdida de datos; tiempo del mensaje	Inyección de fallos dentro de `process()`	0 % pérdida; mensaje <2 s; sistema operativo
Confiabilidad – exportación (Esc. 4)	Tiempo de exportación; error silencioso	`test_export.py`	<3 s; 0 % pérdida; notificación clara (R51/R57/R70)
Usabilidad (Esc. 5)	Tiempo de análisis sin acompañamiento	Prueba de usabilidad con usuarios reales	<5 min; bajas consultas al manual (R42–R44)
Rendimiento – múltiples señales (Esc. 6)	Tiempo de cambio; % de memoria	Benchmark de `DataStore`	<1 s cambio; memoria 70–75 % (R11, R51, R53)
Confiabilidad – archivos corruptos (Esc. 7)	Tiempo del error; continuidad de sesión	`test_fileio_abf/edf.py`	Mensaje <2 s; 0 % pérdida previa (FR-IO-04)
Confiabilidad y Usabilidad (Esc. 8)	Tiempo de actualización; diferencias	Revisión de consistencia entrada-salida	<3 s; 0 diferencias (R73, R03)
Mantenibilidad	% de cobertura automática	`pytest –cov` sobre `core/plugins`	≥80 % de cobertura
Rendimiento – no regresión vs. 1.0	Factor vs. línea base	`compare_report.py`	Nada empeora; render sin vectorizar mejora ≥10×
Portabilidad	Éxito de arranque del `.exe`	Instalación en Windows limpio sin Python	Arranque exitoso, sin configuración manual
Robustez – cancelación (UC-02)	Liberación del hilo tras cancelar	Prueba forzada sobre `TaskOrchestrator`	Liberación garantizada (R46, R70, R74)

6.4. Plan de evidencia

1. **Fixture ABF real y referencias MATLAB.** `test/data/17308005.abf` más los 8 CSV de referencia en `plugins_test/` sustentan la corrección numérica exigida por Extensibilidad (Esc. 1) y por las métricas de Mantenibilidad.

2. **Suite de benchmarks versionada.** `performance_test/`, con resultados persistidos por etiqueta en `perf_results.json` (p. ej. `gammalab1_baseline_2026-08-16`), es la evidencia reproducible de Rendimiento y de la comparación de no regresión frente a 1.0.

3. **Auditorías documentales.** Los informes `reporte_tests.tex` e `informe_rendimiento.tex` (`docs/tests documents/`) registran el estado de la suite de pruebas y los hallazgos de rendimiento.

4. **Pruebas de usabilidad con usuarios reales.** `docs/pruebas-usabilidad/` sustenta el criterio de aceptación de Usabilidad (Esc. 5).
5. **Prueba de instalación limpia.** Arranque del ejecutable generado por PyInstaller en un Windows 10/11 sin Python previamente instalado, como evidencia de Portabilidad.
6. **Registro de cancelación forzada.** La prueba de cancelación forzada sobre el `Task-Orchestrator` bajo carga es la evidencia de Robustez (UC-02).

6.5. Conclusión

Los escenarios de calidad, las reglas de compatibilidad y la tabla de métricas convierten los atributos de calidad de la sección 3.2 en criterios operativos y verificables, no en aspiraciones genéricas. El plan de evidencia de la sección 6.4 sostiene estos criterios con artefactos reproducibles: fixtures reales validados contra una referencia de MATLAB, benchmarks versionados por etiqueta, auditorías documentales, y pruebas de usabilidad y de instalación. Junto con las reglas de versionado de la sección 6.2, esta estructura garantiza que la arquitectura de GammaLab 2.0 evalúe, documente y demuestre sus atributos de calidad de forma reproducible en cada nueva versión del sistema.

7. Anexos

7.1. Anexo A — Relación con el SRS

En particular:

-

7.2. Anexo B — Relación con el SDD

Este anexo resume dicha relación:

-

7.3. Anexo C — Trazabilidad global de artefactos

7.4. Anexo D — Diagramas y tablas de referencia

8. Referencias

Referencias

[1] Python Software Foundation. *Python 3 — The Python Standard Library*. Disponible en: https://docs.python.org/3/.

[2] Riverbank Computing. *PyQt5 Reference Guide*. Disponible en: https://www.riverbankcomputing.com/static/Docs/PyQt5/.

[3] The Qt Company. *Qt 5 — Style Sheets Reference*. Disponible en: https://doc.qt.io/qt-5/stylesheet-syntax.html.

[4] Kitware, Inc. *VTK — Visualization Toolkit Documentation*. Disponible en: https://vtk.org/ y https://docs.vtk.org/.

[5] Kitware, Inc. *vtkChartXY Class Reference*. Disponible en: https://docs.vtk.org/en/latest/api/python/vtkmodules.vtkChartsCore.html#vtkmodules.vtkChartsCore.vtkChartXY.

[6] NumPy Developers. *NumPy Documentation*. Disponible en: https://numpy.org/doc/.

[7] Scott W. Harden. *pyABF — Read and Analyze ABF Files with Python*. Disponible en: https://pyabf.readthedocs.io/.

[8] Holger Nahrstaedt et al. *pyEDFlib — Python library to read/write EDF+/BDF+ files*. Disponible en: https://pyedflib.readthedocs.io/.

[9] Oren Ben-Kiki, Clark Evans, Ingy döt Net. *YAML Ain't Markup Language (YAML) Specification*. Disponible en: https://yaml.org/spec/.

[10] P. B. Kruchten. *The 4+1 View Model of Architecture*. IEEE Software, vol. 12, no. 6, pp. 42–50, 1995. DOI: https://doi.org/10.1109/52.469759.

[11] Len Bass, Paul Clements, Rick Kazman. *Software Architecture in Practice* (3rd ed.). Addison–Wesley, 2012. ISBN 978-0-321-81573-6.

[12] Paul Clements, Felix Bachmann, Len Bass, David Garlan, James Ivers, Reed Little, Robert Nord, Judith Stafford. *Documenting Software Architectures: Views and Beyond* (2nd ed.). Addison–Wesley, 2010. ISBN 978-0-321-55268-6.

[13] Rick Kazman, Mark Klein, Paul Clements. *ATAM: Method for Architecture Evaluation*. CMU/SEI-2000-TR-004, Software Engineering Institute, 2000.

[14] Bob Kemp, Alpo Värri, Agostinho Rosa, Kim D. Nielsen, John Gade. *A simple format for exchange of digitized polygraphic recordings*. Electroencephalography and Clinical Neurophysiology, 82(5), 1992, pp. 391–393. DOI: https://doi.org/10.1016/0013-4694(92)90009-7.

[15] P. L. Nunez, R. Srinivasan. *Electric Fields of the Brain: The Neurophysics of EEG* (2nd ed.). Oxford University Press, 2006.

[16] Equipo Gamma Lab. *Software Architecture Document (SAD), v1.0*. Este documento.

[17] Object Management Group. *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. OMG Doc. formal/17-12-05, dic. 2017. Disponible en: https://www.omg.org/spec/UML/2.5.1/.

Nota: Las URLs señaladas se incluyen para consulta rápida. En publicación impresa, usar DOI/ISBN cuando aplique.